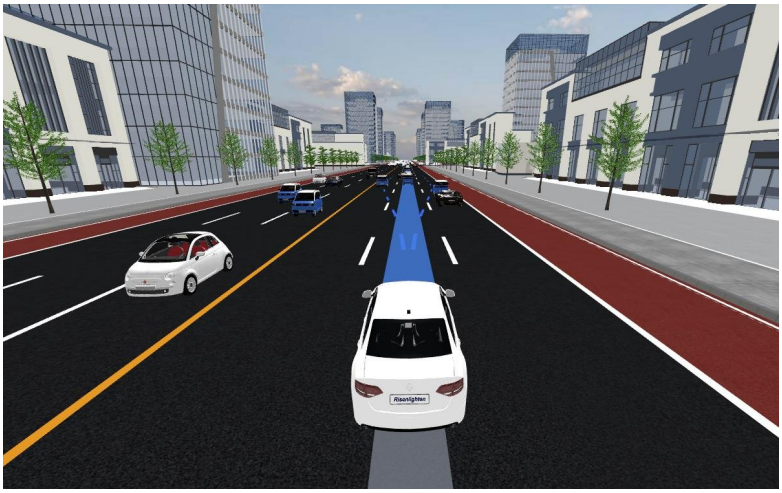
HESTIM
ENGINEERING & BUSINESS SCHOOL

Filière : GÉNIE INFORMATIQUE

Projet PACTE

CLASSIFICATION D'OBJETS ROUTIERS PAR MACHINE LEARNING POUR LES VÉHICULES AUTONOMES

AutoMaze AI, Livrable 1 : dossier Go/No-Go



Réalisé par : LUSAMOTE KIMFUTA Claudia
MOUPIGA TOMBE Elisia
Issa D DEMBELE

Encadrante : Mme HAIDRAR

Année universitaire : 2026 – 2027

Table des matières

INTRODUCTION GÉNÉRALE **8**

1 Présentation du projet et cahier des charges **9**

1.1 Introduction . . . 9

1.2 Présentation et contexte . . . 9

1.3 Problématique . . . 9

1.4 Objectifs du projet . . . 10

1.4.1 Objectif général . . . 10

1.4.2 Objectifs spécifiques . . . 10

1.5 Périmètre fonctionnel . . . 11

1.6 Besoins fonctionnels . . . 12

1.7 Besoins non fonctionnels . . . 12

1.8 Contraintes techniques et académiques . . . 13

1.9 Méthodologie et planning prévisionnel . . . 14

1.10 Conclusion . . . 15

2 Analyse fonctionnelle et conception du système **17**

2.1 Introduction . . . 17

2.2 Identification des acteurs . . . 17

2.3 Description des cas d'utilisation . . . 18

2.4 Scénarios de fonctionnement . . . 18

2.5 Diagramme de cas d'utilisation . . . 19

2.6 Diagramme d'activité . . . 20

2.7 Diagramme de séquence . . . 21

2.8 Diagramme de composants . . . 22

2.9 Description des interactions entre modules . . . 23

2.10 Conclusion . . . 23

3 Étude et préparation des données **24**

3.1 Introduction . . . 24

3.2 Étude des datasets disponibles 24
3.3 Comparaison et justification du dataset retenu 24
3.3.1 Source et description du dataset retenu 26
3.4 Acquisition des données à partir des annotations 26
3.4.1 Pourquoi récupérer les annotations plutôt que toutes les images 26
3.4.2 Comment les données sont récupérées 27
3.4.3 Exemple d'annotation 28
3.4.4 Bénéfices de l'utilisation des annotations 29
3.5 Présentation des classes 30
3.6 Analyse quantitative des données 31
3.7 Analyse qualitative des images 32
3.8 Nettoyage des données 32
3.9 Redimensionnement et normalisation 32
3.10 Séparation entraînement, validation et test 33
3.11 Conclusion 33

4 Architecture technique et conception des modules 35
4.1 Introduction 35
4.2 Architecture globale d'AutoMaze AI 35
4.3 Environnement de simulation 36
4.4 Acquisition des images par caméra virtuelle 36
4.4.1 Association d'une image à chaque objet 37
4.5 Module de classification par Machine Learning 37
4.6 Module de localisation et association des objets 38
4.7 Représentation de l'environnement et des obstacles 39
4.8 Algorithme de planification Dijkstra 39
4.8.1 Justification du choix de Dijkstra 39
4.9 Système de décision 40
4.10 Contrôle du véhicule virtuel 41
4.11 Communication et intégration des modules 41
4.12 Conclusion 41

5 Méthodologie expérimentale et évaluation 43
5.1 Introduction 43

5.2 Extraction des caractéristiques 43
5.3 Présentation des algorithmes SVM, Random Forest et k-NN 43
5.4 Protocole expérimental 44
5.5 Métriques d'évaluation 44
5.6 Méthode de comparaison des modèles 45
5.7 Critères de sélection du modèle final 45
5.8 Protocole de validation dans le simulateur 46
5.9 Conclusion 46

6 Étude de faisabilité et validation Go/No-Go 47
6.1 Introduction 47
6.2 Faisabilité technique 47
6.3 Faisabilité matérielle et logicielle 47
6.4 Identification des risques 48
6.5 Preuves de faisabilité disponibles 49
6.5.1 Déroulement des comparaisons préliminaires 49
6.6 Planning de réalisation 51
6.7 Critères de décision Go/No-Go 52
6.8 Conclusion 53

CONCLUSION GÉNÉRALE 54

BIBLIOGRAPHIE 55

A Sources des illustrations 56

B Usage de l'intelligence artificielle 57

Liste des figures

1 Détection d'objets par réseau à régions (R-CNN), hors du périmètre de notre projet 11

2 Flux de développement, d'entraînement et d'intégration du modèle (lire COCO 2017, classes complétées par le camion) 15

3 Scénarios de démonstration (illustrations externes) 19

4 Diagramme de cas d'utilisation 20

5 Diagramme d'activité d'un épisode de conduite 21

6 Diagramme de séquence du traitement d'un obstacle 22

7 Diagramme de composants 22

8 Extrait réel du fichier d'annotations de COCO `train2017` (segmentation raccourcie) 28

9 De l'annotation au recadrage : boîte de l'annotation 1193435 sur la photo d'origine, puis recadrage de 64 × 64 px (photo COCO 000000384827, licence CC BY-NC 2.0) 29

10 Illustration des six classes par des modèles 3D (le modèle apprend sur des photos réelles de COCO) 31

11 Répartition des objets par classe avant filtrage 31

12 Recadrages de 64 × 64 px du jeu d'entraînement, une ligne par classe 33

13 Architecture fonctionnelle globale d'AutoMaze AI 35

14 Exemples d'environnements virtuels de conduite ayant servi d'inspiration (sources externes) 36

15 Notre simulateur en vue carte : itinéraire, obstacles et dernière prédiction (carte © contributeurs OpenStreetMap) 36

16 Comparaison préliminaire du meilleur réglage de chaque famille sur la validation . 51

17 Planning prévisionnel du projet 51

18 Répartition indicative des efforts (proposition et non mesure du temps déjà passé) 52

Liste des tableaux

1 Répartition des rôles au sein de l'équipe 9
2 Objectifs spécifiques et mesures de réussite 10
3 Délimitation du périmètre par module 11
4 Exigences fonctionnelles consolidées 12
5 Ressources nécessaires au projet 14

6 Résumé des acteurs et de leurs rôles 18
7 Description des cas d'utilisation 18
8 Interactions entre les modules du système 23

9 Comparaison des datasets étudiés 24
10 Comparaison des datasets selon nos critères 25
11 Fiche descriptive du dataset COCO 2017 26
12 Volume des données stockées sur Google Drive 28
13 Champs de l'annotation utilisés dans le projet 29
14 Nombre d'objets disponibles par classe dans COCO `train2017` 31
15 Répartition des images par classe et par jeu 33

16 Exemple d'association entre position et classe prédite 38
17 Comparaison des algorithmes de plus court chemin envisagés 40
18 Règles de décision selon la situation rencontrée 40
19 Points d'échange entre les modules 41

20 Métriques d'évaluation retenues 45
21 Mesures relevées dans le simulateur 46

22 Outils et bibliothèques retenus 48
23 Risques identifiés et parades prévues 48
24 Preuves de faisabilité disponibles au 6 novembre 2026 49
25 Réglages testés lors des comparaisons préliminaires (jeu de validation) 50
26 Critères de décision Go/No-Go 52

27 Sources des illustrations 56

INTRODUCTION GÉNÉRALE

Dans un contexte où les véhicules autonomes occupent une place de plus en plus importante dans la réflexion sur la mobilité de demain, la **perception de l'environnement routier** représente un enjeu majeur. Pour circuler sans conducteur, un véhicule doit être capable de reconnaître les usagers qui l'entourent, car un piéton qui traverse et un camion à l'arrêt n'appellent pas la même réaction. Une erreur à cette étape peut ainsi entraîner une mauvaise décision de conduite.

L'objectif principal du **projet PACTE** est de concevoir et de développer un système de **classification d'objets routiers par Machine Learning**, capable de reconnaître six catégories d'usagers de la route à partir d'images : la personne, le vélo, la voiture, la moto, le bus et le camion. Le modèle est entraîné de zéro, sans réseau préentraîné ni détecteur d'objets, conformément aux règles du cours.

Ce projet s'inscrit dans une démarche de professionnalisation visant à mobiliser les compétences acquises en **traitement de données**, en **apprentissage automatique** et en **conception logicielle**. Il permet de mettre en pratique la préparation d'un jeu de données, l'extraction de caractéristiques visuelles et la comparaison de plusieurs classifieurs (**SVM**, **Random Forest** et **k-NN**), ainsi que l'analyse rigoureuse de leurs résultats.

Au-delà de l'aspect académique, le projet intègre une **démonstration de conduite autonome** : un véhicule virtuel circule dans les rues de Casablanca et adapte son trajet en fonction des prédictions de notre modèle. Cette intégration permet de montrer concrètement l'utilité de la classification dans un véhicule autonome, sans devenir le sujet principal du projet.

Ce dossier, qui prépare la décision de Go/No-Go (15 % de la note finale), est organisé en six chapitres. Le premier présente le projet et son cahier des charges, le deuxième l'analyse fonctionnelle et la conception du système, et le troisième l'étude et la préparation des données. Le quatrième détaille l'architecture technique des modules, le cinquième la méthodologie expérimentale, et le sixième l'étude de faisabilité ainsi que les critères de la décision Go/No-Go.

Chapitre 1 : Présentation du projet et cahier des charges

1.1- Introduction

Avant toute réalisation technique, il est essentiel de définir le cadre général du projet, ses objectifs, les besoins à satisfaire et la méthode à suivre. Le cahier des charges constitue la base du travail en précisant les aspects fonctionnels, organisationnels et techniques qui orienteront la suite du projet. Dans le cadre du **projet PACTE**, il permet d'identifier les attentes liées au sujet, de les traduire en exigences concrètes et de déterminer les contraintes à respecter. Ce chapitre présente donc le projet, sa problématique, ses objectifs, son périmètre, les besoins fonctionnels et non fonctionnels ainsi que la méthodologie et le planning prévisionnel.

1.2- Présentation et contexte

Ce projet a pour objectif de concevoir un système capable de reconnaître, à partir de l'image d'un objet, six usagers de la route : la **personne**, le **vélo**, la **voiture**, la **moto**, le **bus** et le **camion**. Pour chaque image, le modèle renvoie une classe ainsi qu'un **score de confiance**. Une fois entraîné, il est testé sur des images qu'il n'a jamais vues, puis utilisé par un véhicule virtuel qui circule dans les vraies rues de Casablanca.

Le projet place l'équipe dans la position d'une cellule de développement chargée de mettre en œuvre la brique de perception d'un véhicule autonome. Notre équipe est composée de trois étudiants, chacun étant responsable d'un rôle précis.

Tableau 1 *: Répartition des rôles au sein de l'équipe*

Rôle	Responsable	Missions
Données	**[Nom]**	Exploration, prétraitement, recadrages et découpage des données
Modèles	**[Nom]**	Extraction des caractéristiques, entraînement des classifieurs et suivi des essais
Évaluation et intégration	**[Nom]**	Évaluation finale, analyse des erreurs, export du modèle et lien avec le simulateur

1.3- Problématique

Dans la plupart des systèmes de conduite autonome, la reconnaissance des usagers de la route repose sur des modèles très volumineux, entraînés sur des millions d'images et sur des machines puissantes. Dans un cadre académique, avec un **jeu de données de taille limitée**, **sans poids préentraînés** et **sans GPU dédié**, la question de la fiabilité d'un tel modèle se pose naturellement.

Le projet AutoMaze AI vise à répondre à la question suivante :

Un modèle entraîné de zéro sur un jeu de données de taille limitée peut-il reconnaître les usagers de la route de manière suffisamment fiable pour guider les décisions d'un véhicule autonome virtuel ?

De cette question découlent trois interrogations complémentaires. Il s'agit d'abord de déterminer quel classifieur, parmi SVM, Random Forest et k-NN, obtient les meilleurs résultats sur les mêmes caractéristiques. Il faut ensuite identifier les classes les plus difficiles à reconnaître et comprendre pourquoi. Enfin, il est important de mesurer les conséquences d'une erreur du modèle sur le comportement du véhicule, notamment lorsqu'un piéton n'est pas reconnu.

1.4- Objectifs du projet

1.4.1- Objectif général

L'objectif principal du projet est de **développer et d'évaluer un classifieur d'objets routiers entraîné de zéro**, puis de montrer comment ses prédictions peuvent guider un **véhicule autonome virtuel** jusqu'à sa destination.

1.4.2- Objectifs spécifiques

Pour atteindre cet objectif général, plusieurs objectifs spécifiques ont été définis :

Tableau 2 *: Objectifs spécifiques et mesures de réussite*

N°	Objectif spécifique	Mesure de réussite
OS1	Décrire le jeu de données retenu	Provenance, volume, répartition par classe, résolution et qualité documentées
OS2	Préparer les images	Nettoyage, recadrage, normalisation et découpage sans fuite de données
OS3	Comparer SVM, Random Forest et k-NN	Accuracy, précision, rappel, F1 et matrice de confusion, calculés sur les mêmes données
OS4	Analyser les classes difficiles	Paires de classes confondues et images mal classées expliquées
OS5	Prédire la classe de nouvelles images	Affichage du type « Image → Voiture (92 %) »
OS6	Intégrer le modèle au véhicule virtuel	Chaque prédiction déclenche une décision, et les erreurs graves sont comptées

Pour juger nos modèles, nous retenons principalement le **F1 macro**, qui accorde le même poids à chaque classe. Aucune valeur cible n'est fixée à l'avance : la référence sera le meilleur modèle classique, comparé à des scores planchers (section 5.5).

1.5- Périmètre fonctionnel

Afin d'éviter que le projet ne devienne trop vaste, nous distinguons les **fonctionnalités principales**, qui relèvent du Machine Learning et constituent le cœur scientifique du projet, des **fonctionnalités d'intégration**, qui servent à la démonstration finale.

Tableau 3 *: Délimitation du périmètre par module*

Module	Responsabilité	Importance
Dataset	Fournir les images des objets routiers	Essentielle
Prétraitement	Préparer les images pour l'apprentissage	Essentielle
Classification ML	Identifier la classe d'un objet	Essentielle
Évaluation	Comparer les classifieurs	Essentielle
Simulateur	Fournir un environnement de démonstration	Intégration
Localisation	Fournir les coordonnées des objets	Intégration
Planification	Calculer le chemin le plus court	Intégration
Décision	Choisir le comportement face aux obstacles	Intégration
Contrôle	Commander le véhicule virtuel	Intégration

Les fonctionnalités d'intégration sont nécessaires à la démonstration, mais nous veillerons à ce qu'elles ne prennent pas le dessus sur les expérimentations de Machine Learning demandées par le sujet.

La détection d'objets reste hors périmètre. Notre modèle ne recherche pas la position des objets dans une image complète, ce qui relèverait d'un détecteur comme YOLO ou R-CNN (figure 1), interdit par les règles du cours. Il reçoit l'image d'un objet déjà isolé et en détermine la nature. Dans le simulateur, c'est la position connue de chaque objet qui remplace l'étape de détection. La vidéo et la physique réaliste du véhicule sont également exclues du périmètre.

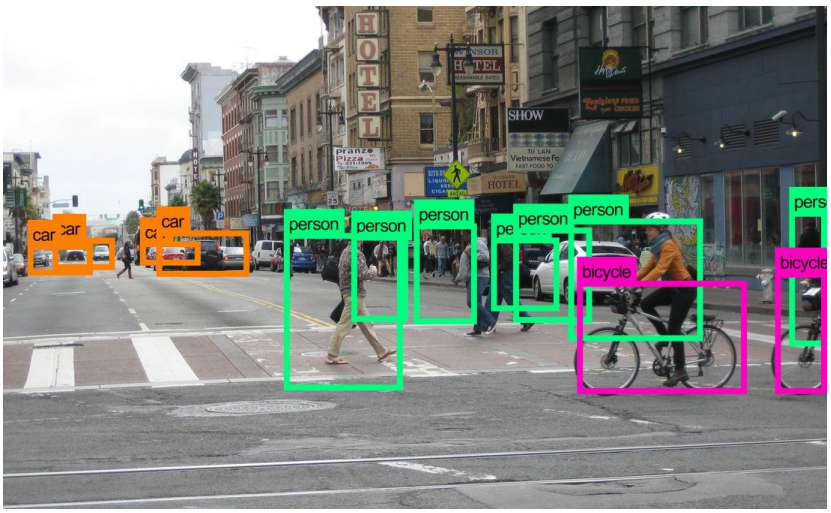


Figure 1 *: Détection d'objets par réseau à régions (R-CNN), hors du périmètre de notre projet*

1.6- Besoins fonctionnels

Les besoins fonctionnels regroupent l'ensemble des fonctionnalités que le système doit offrir. Les exigences EF01 à EF06 reprennent les attentes du sujet, tandis que les exigences EF07 à EF14 concernent l'intégration dans le véhicule virtuel. Pour chacune d'elles, nous avons défini un critère d'acceptation qui permettra de vérifier qu'elle est remplie.

Tableau 4 *: Exigences fonctionnelles consolidées*

ID	Exigence	Critère d'acceptation
EF01	Charger et explorer le dataset routier	Effectifs par classe, tailles et exemples présentés (notebook 01)
EF02	Prétraiter les images	Images filtrées, recadrées, redimensionnées et normalisées, découpage sans photo partagée (notebook 02)
EF03	Extraire les caractéristiques visuelles	Descripteurs HOG calculés pour toutes les images (notebook 03)
EF04	Entraîner plusieurs modèles de classification	SVM, Random Forest et k-NN entraînés sur les mêmes caractéristiques
EF05	Évaluer et comparer les modèles	Tableau commun : accuracy, précision, rappel, F1 et matrices de confusion
EF06	Prédire la classe d'un nouvel objet	Une image inconnue donne une classe et un score affichés
EF07	Récupérer les coordonnées des objets dans le simulateur	Chaque obstacle possède un tronçon de rue et une position sur ce tronçon
EF08	Associer les prédictions aux objets virtuels	Chaque prédiction est rattachée à l'obstacle dont provient l'image
EF09	Représenter les obstacles dans l'environnement	Tronçons bloqués ou pénalisés selon la décision
EF10	Calculer le chemin vers la destination avec Dijkstra	Plus court chemin trouvé sur la carte et vérifié par des tests
EF11	Adapter le trajet en présence d'un obstacle	Nouveau calcul après chaque obstacle bloquant
EF12	Décider de ralentir, s'arrêter ou contourner	Une action par classe prédite, et un arrêt prudent si le score est trop faible
EF13	Commander le véhicule virtuel	Le véhicule avance, ralentit, s'arrête, fait demi-tour ou change de rue
EF14	Afficher les prédictions et les décisions du véhicule	Image, classe, score et action visibles à l'écran

1.7- Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité et les contraintes que le système doit respecter. Ils ne concernent pas directement les fonctionnalités visibles, mais influencent fortement la fiabilité des résultats et la crédibilité du projet.

- **Entraînement de zéro**
  Aucun YOLO, aucun détecteur et aucun poids préentraîné ne doivent être utilisés dans notre modèle.

- **Fiabilité de l'évaluation**
  Le découpage doit se faire par photo d'origine, et le jeu de test ne doit être utilisé qu'une seule fois, lors de l'évaluation finale.

- **Reproductibilité**
  Les graines aléatoires doivent être fixées, les essais notés, et les notebooks exécutables de bout en bout.

- **Cohérence du prétraitement**
  La taille des images, l'ordre des couleurs et la normalisation doivent être identiques dans les notebooks et dans le simulateur.

- **Séparation entre vérité terrain et perception**
  Les décisions du véhicule ne doivent jamais utiliser la vraie classe connue du simulateur, mais uniquement la classe prédite par le modèle.

- **Honnêteté des résultats**
  Aucune métrique ne doit être inventée. Sans modèle chargé, l'interface doit afficher « SIMULATION — modèle réel non chargé ».

- **Respect des licences**
  Les sources des données (COCO), de la carte (OpenStreetMap, licence ODbL) et des illustrations doivent être citées.

- **Temps de réponse**
  Une prédiction doit prendre quelques millisecondes sur un processeur ordinaire, afin de garantir une démonstration fluide.

1.8- Contraintes techniques et académiques

Le projet doit respecter plusieurs contraintes, liées à la fois au cadre du cours et aux moyens dont dispose l'équipe :

- **Règles du cours**
  L'utilisation de YOLO, de détecteurs et de poids préentraînés est interdite pour notre modèle. Les classifieurs à comparer sont ceux du sujet : SVM, Random Forest et k-NN.

- **Calendrier**
  Le projet s'organise autour de trois échéances fixes : la séance 1 le 4 novembre, la séance 2 le 25 novembre et la soutenance le 16 décembre 2026.

- **Matériel**
  Aucun membre de l'équipe ne dispose d'un GPU. Les traitements doivent donc pouvoir s'exécuter sur un processeur ordinaire, avec le GPU gratuit de Google Colab en appui.

- **Budget**

Le projet ne nécessite aucun budget, car l'ensemble des outils et des données retenus sont gratuits.

- **Licences**
  Les images de COCO conservent leur licence Flickr et ne sont donc pas redistribuées dans le dépôt Git.

Tableau 5 *: Ressources nécessaires au projet*

Ressource	Détail	Solution
Données	Images des 6 classes, étiquettes fiables, licence compatible	COCO 2017, à valider par l'encadrante
Calcul	Entraînement des modèles	Processeur local, Google Colab en appui
Stockage	Données, poids et figures partagés	Dossier Google Drive partagé
Poste local	Fenêtre du simulateur, que Colab ne peut pas ouvrir	Disponible
Encadrement	Validation du dataset et du périmètre	Section 6.7

1.9- Méthodologie et planning prévisionnel

Pour la conduite de ce projet, nous avons retenu une démarche en **deux phases**, illustrée par la figure 2. La première phase consiste à développer et à entraîner le modèle dans des notebooks versionnés sur GitHub. La seconde consiste à charger le modèle exporté dans le simulateur, sans jamais l'y réentraîner.

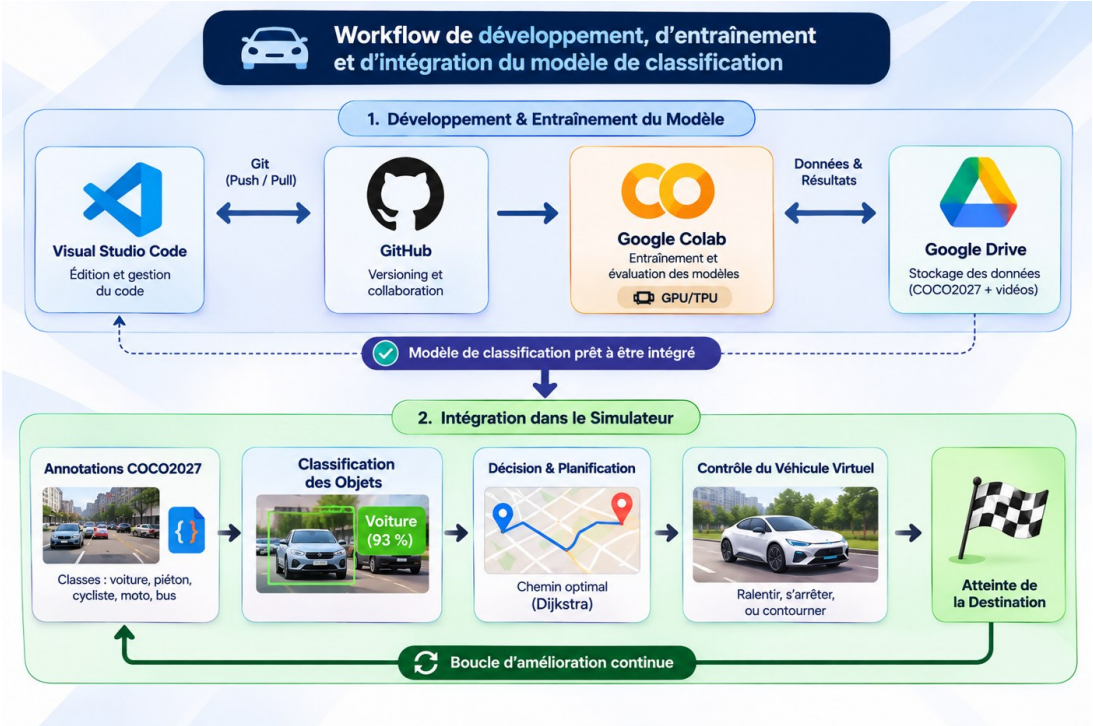


Figure 2 *: Flux de développement, d'entraînement et d'intégration du modèle (lire COCO 2017, classes complétées par le camion)*

Cette démarche s'organise en quatre grandes étapes :

1. **Données (notebooks 01 et 02)** : acquisition, exploration, nettoyage, recadrage et découpage des images.
2. **Modèles (notebook 03)** : extraction des descripteurs HOG, puis entraînement et réglage de SVM, Random Forest et k-NN sur le jeu de validation.
3. **Évaluation (notebook 05)** : mesure unique sur le jeu de test, analyse des erreurs et prédiction sur de nouvelles images.
4. **Intégration** : export du modèle, branchement dans le simulateur et comparaison sur plusieurs épisodes de conduite.

Sur le plan de l'organisation, une seule personne modifie un notebook à la fois, et chaque notebook est relu par un autre membre avant chaque séance. Les décisions importantes sont consignées dans un journal de bord. Le planning détaillé est présenté à la section 6.6.

1.10- Conclusion

Ce premier chapitre a permis de définir le **cadre général du projet AutoMaze AI** en précisant son contexte, sa problématique, ses objectifs et son périmètre. L'élaboration du cahier des charges a constitué une étape essentielle pour traduire les attentes du sujet en exigences vérifiables, tout en distinguant clairement le travail de Machine Learning, qui reste au cœur du projet, des fonctionnalités d'intégration. Cette analyse préliminaire servira de base au chapitre suivant, consacré

à l’analyse fonctionnelle et à la conception du système.

Chapitre 2 : Analyse fonctionnelle et conception du système

2.1- Introduction

Après avoir défini les objectifs, le périmètre et les contraintes du projet dans le cahier des charges, cette deuxième étape vise à analyser le **fonctionnement attendu du système** afin de poser les bases de sa conception. L'analyse fonctionnelle assure la cohérence entre les attentes du sujet et la structure technique de la solution. Dans un premier temps, nous identifierons les acteurs et les cas d'utilisation, puis nous décrirons les scénarios de démonstration. Nous formaliserons ensuite le comportement du système à travers les diagrammes **UML** de cas d'utilisation, d'activité, de séquence et de composants.

2.2- Identification des acteurs

L'identification des acteurs constitue une étape essentielle de l'analyse fonctionnelle. Elle permet de déterminer les différentes parties qui interagissent avec le système, chacune ayant un rôle et des responsabilités spécifiques.

1. Équipe projet
Rôle : l'équipe conçoit et entraîne le modèle de classification.
Responsabilités :

- Préparer les données et réaliser leur découpage.
- Entraîner et comparer les classifieurs.
- Évaluer le modèle final et analyser ses erreurs.

2. Utilisateur de la démonstration
Rôle : l'utilisateur interagit avec le système pour observer son fonctionnement.
Responsabilités :

- Classer une nouvelle image.
- Lancer un épisode de conduite dans le simulateur.

3. Encadrante et jury
Rôle : l'encadrante valide les choix du projet et suit son avancement, tandis que le jury évalue les résultats.
Responsabilités :

- Valider le jeu de données et le périmètre du projet.
- Consulter les résultats, le rapport et la démonstration.

À ces acteurs humains s'ajoutent deux acteurs système : le **simulateur**, qui fournit l'environne-

ment, les positions des objets et leurs images, et le **modèle ML**, qui renvoie la classe et le score d'une image.

Tableau 6 *: Résumé des acteurs et de leurs rôles*

Acteur	Type	Rôle principal
Équipe projet	Humain	Préparation des données, entraînement et évaluation des modèles
Utilisateur de la démo	Humain	Classification d'images et lancement d'épisodes de conduite
Encadrante	Humain	Validation des choix et suivi de l'avancement
Jury	Humain	Évaluation des résultats, du rapport et de la démonstration
Simulateur	Système	Environnement, positions et images des objets
Modèle ML	Système	Classe et score de chaque image

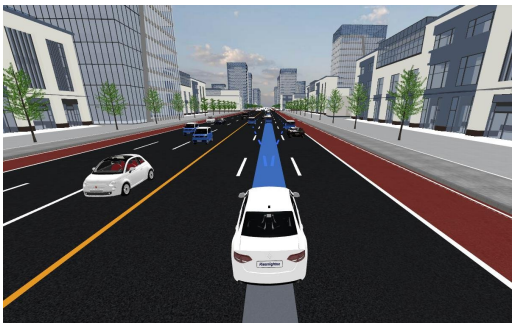
2.3- Description des cas d'utilisation

Tableau 7 *: Description des cas d'utilisation*

ID	Cas d'utilisation	Acteur	Résultat attendu
UC1	Préparer les données	Équipe	Recadrages filtrés, normalisés et découpés en entraînement, validation et test
UC2	Entraîner et comparer les classifieurs	Équipe	Tableau comparatif de SVM, Random Forest et k-NN sur la validation
UC3	Évaluer le modèle final	Équipe, encadrante	Scores sur le test, matrice de confusion et analyse des erreurs
UC4	Classer une nouvelle image	Utilisateur	« photo.jpg → Voiture (92 %) »
UC5	Lancer une démonstration de conduite	Utilisateur, jury	Le véhicule rejoint sa destination en réagissant aux obstacles
UC6	Comparer les classifieurs dans le simulateur	Équipe	Collisions, fautes graves et décisions inadaptées sur plusieurs épisodes
UC7	Consulter les résultats et le rapport	Encadrante, jury	Notebooks commentés, figures et rapport

2.4- Scénarios de fonctionnement

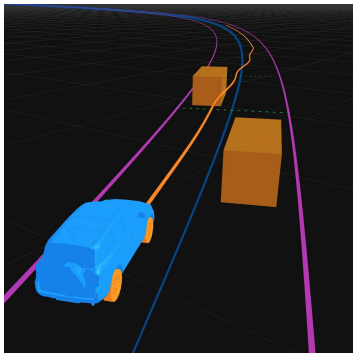
Ces scénarios serviront à la fois à concevoir les tests et à montrer ce que le système final devra accomplir. Les illustrations de la figure 3 proviennent d'autres simulateurs et servent uniquement d'inspiration. Une capture de notre propre simulateur est présentée à la section 4.3.



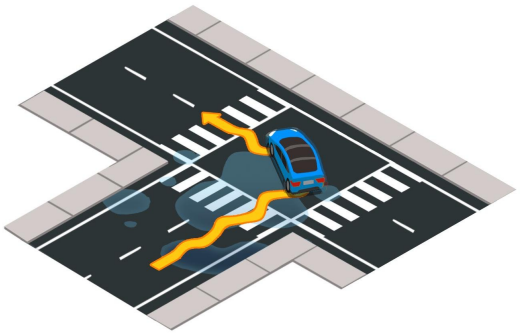
(a) Scénario 1 : route libre



(b) Scénario 2 : présence d'un piéton



(c) Scénario 3 : route bloquée



(d) Scénario 4 : contournement

Figure 3 *: Scénarios de démonstration (illustrations externes)*

- **Scénario 1 : route libre (cas nominal)**
  Aucun obstacle ne se trouve sur l'itinéraire. Le véhicule suit le chemin calculé par Dijkstra jusqu'à sa destination.
- **Scénario 2 : présence d'un piéton**
  Un objet classé « personne » se trouve sur l'itinéraire, à moins de 35 m. Le véhicule s'arrête, attend que la personne traverse, puis repart sur le même trajet.
- **Scénario 3 : route bloquée**
  Un objet classé « voiture », « bus » ou « camion » occupe le tronçon. Celui-ci est marqué comme bloqué dans le graphe, et Dijkstra recherche un autre chemin.
- **Scénario 4 : contournement**
  Un nouveau chemin existe. Le véhicule change de rue, ou fait demi-tour si nécessaire, puis reprend sa progression vers la destination.

Deux cas particuliers sont également prévus. Si le score de confiance est trop faible, la classe devient « Inconnu » et le véhicule effectue un arrêt prudent. S'il n'existe plus aucun chemin, le véhicule reste arrêté, et l'épisode est déclaré « bloqué » au bout de 30 pas.

2.5- Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation du projet *AutoMaze AI* illustre les différentes interactions entre les acteurs et les fonctionnalités principales du système. L'équipe projet intervient sur toute la chaîne de Machine Learning, de la préparation des données à l'évaluation. L'utilisateur de la dé-

monstration peut classer une image ou lancer un épisode de conduite, tandis que l'encadrante et le jury consultent les résultats. Le diagramme met également en évidence le lien d'**inclusion** entre la démonstration de conduite et la classification d'une image, puisque chaque obstacle rencontré doit être classé.

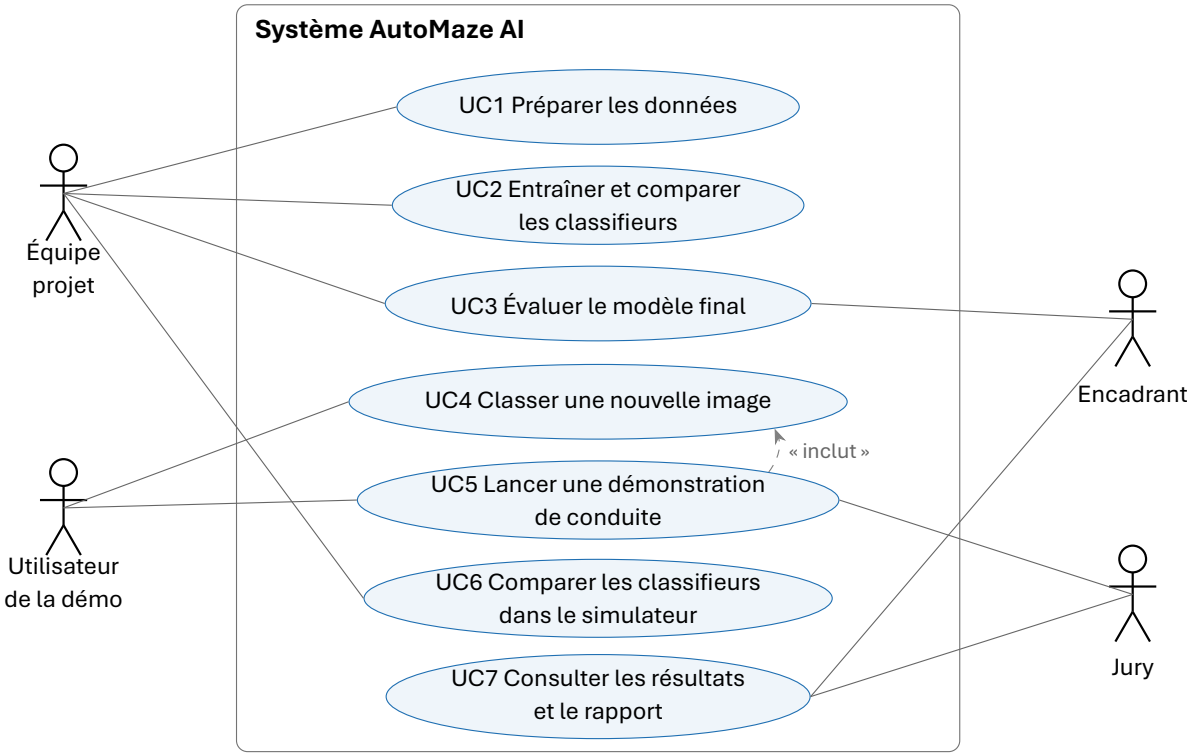


Figure 4 *: Diagramme de cas d'utilisation*

2.6- Diagramme d'activité

Le diagramme d'activité décrit le déroulement d'un **épisode de conduite**, depuis le calcul de l'itinéraire jusqu'à l'arrivée. À chaque pas, le véhicule avance puis vérifie s'il a atteint sa destination. Lorsqu'un obstacle se trouve à moins de 35 m, son image est transmise au modèle, et la classe prédite détermine l'action à appliquer. Les coûts des tronçons sont alors mis à jour, et le trajet est recalculé. Ce diagramme met en évidence la coordination entre la perception, la décision et la planification.

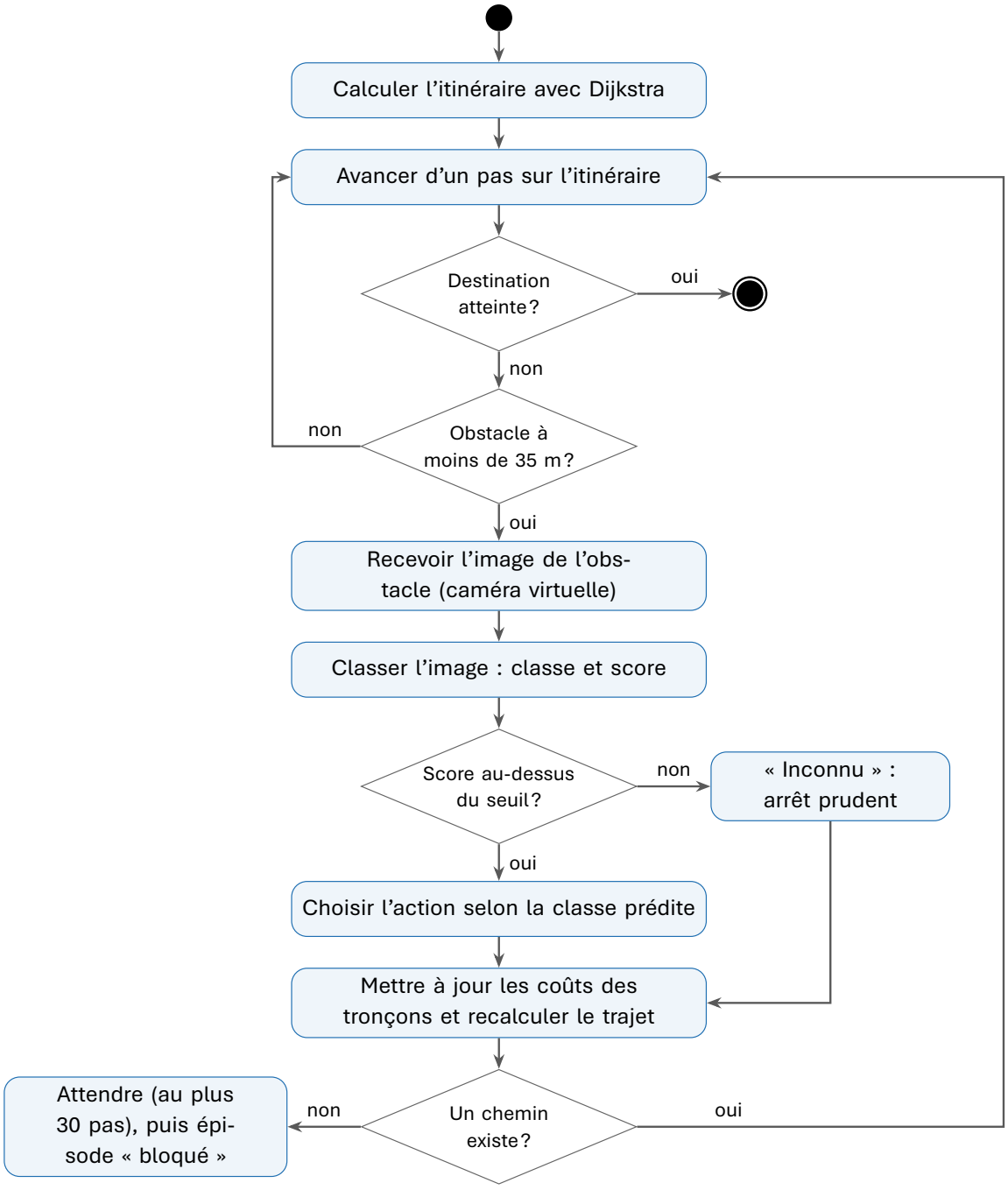


Figure 5 *: Diagramme d'activité d'un épisode de conduite*

2.7- Diagramme de séquence

Le diagramme de séquence décrit les échanges entre les modules lorsque le véhicule rencontre un obstacle. Le moteur de simulation demande l'image de l'obstacle à la caméra virtuelle, puis la transmet au classifieur, qui renvoie la classe, le score et les six probabilités. La classe prédite et la position de l'obstacle sont ensuite envoyées au système de décision, qui modifie le graphe, avant que le planificateur ne recalcule le trajet. Il est important de noter que la **vraie classe de l'objet n'apparaît dans aucun message** : le système de décision ne reçoit que la classe prédite.

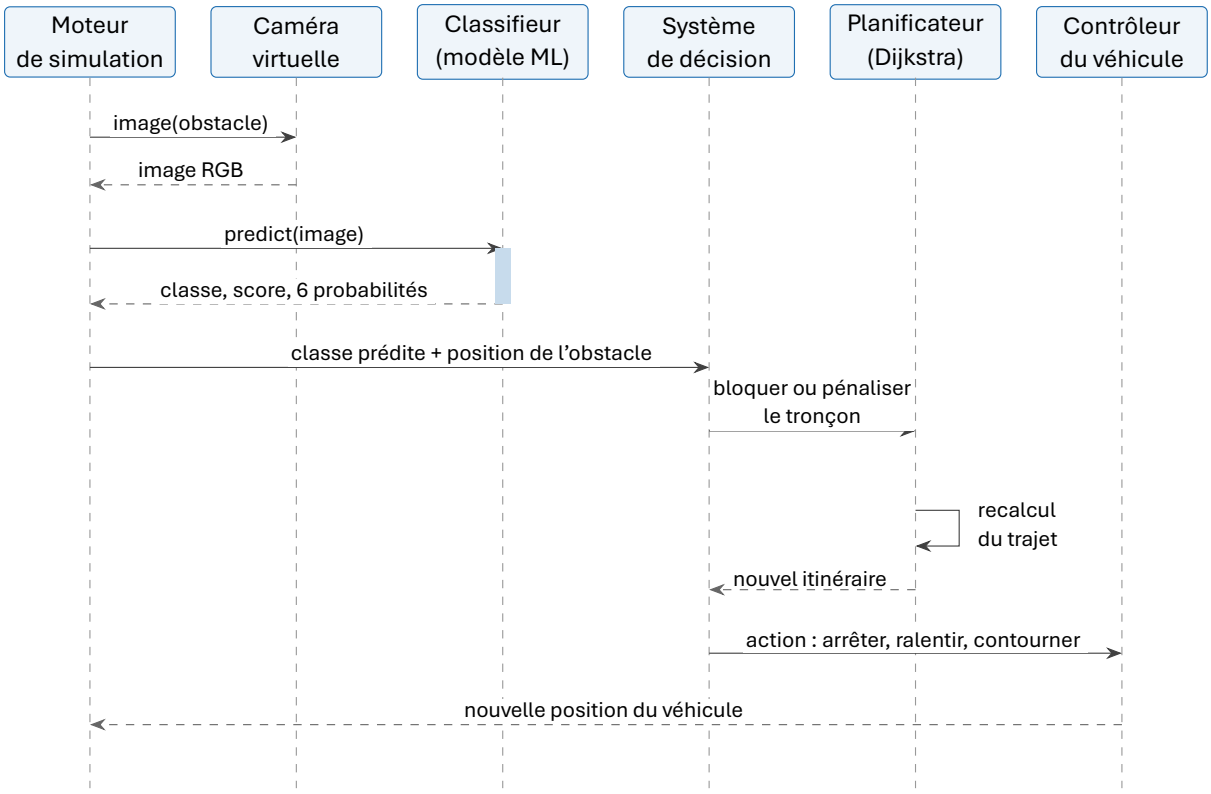


Figure 6 *: Diagramme de séquence du traitement d'un obstacle*

2.8- Diagramme de composants

Le diagramme de composants présente l'organisation logicielle du projet. La chaîne de Machine Learning, réalisée dans les notebooks, produit les poids du modèle et le fichier `metadata.json`. Le simulateur charge ces fichiers à travers son module de perception, et les différents modules communiquent grâce aux contrats partagés définis dans `contracts.py`.

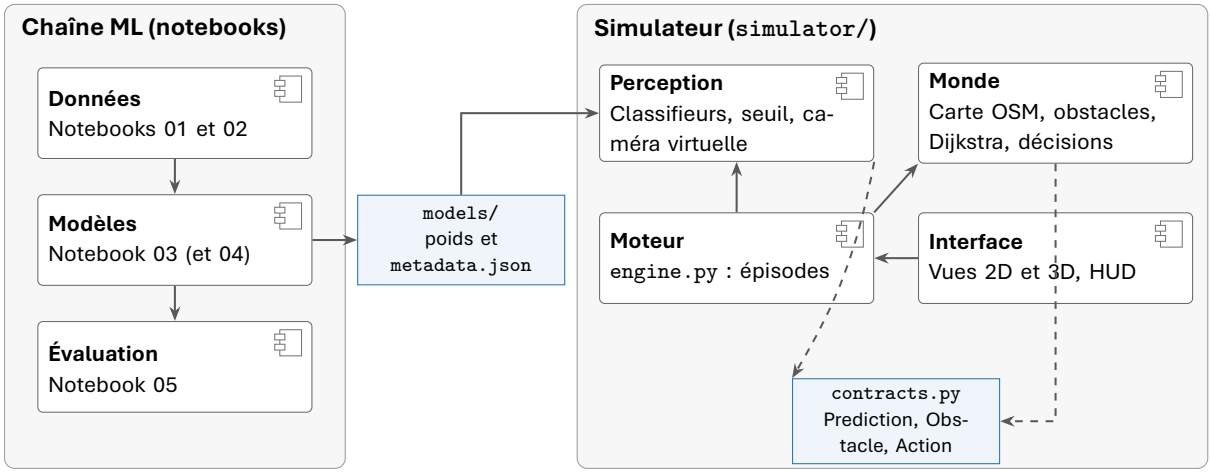


Figure 7 *: Diagramme de composants*

2.9- Description des interactions entre modules

Tableau 8 *: Interactions entre les modules du système*

De	Vers	Information échangée
Notebooks	Simulateur	Poids du modèle et `metadata.json` : classes, taille d'entrée, ordre des couleurs, normalisation et seuil
Moteur	Caméra virtuelle	Obstacle rencontré, et en retour une image RGB
Moteur	Classifieur	Image RGB, et en retour une prédiction : classe, score, 6 probabilités et temps de calcul
Moteur	Décision	Classe prédite et tronçon de l'obstacle, et en retour l'action à appliquer
Décision	Planificateur	Tronçons bloqués ou pénalisés, et en retour un nouvel itinéraire
Moteur	Contrôleur	Itinéraire, attente ou ralentissement, et en retour la position du véhicule
Moteur	Interface	État de l'épisode, dernière image, prédiction et décision

2.10- Conclusion

Ce chapitre a permis de formaliser le **fonctionnement attendu du système AutoMaze AI** à travers l'identification des acteurs, la description des cas d'utilisation et des scénarios de démonstration, ainsi que plusieurs diagrammes UML. Ces représentations ont contribué à clarifier les échanges entre la perception, la décision et la planification, tout en garantissant que les décisions du véhicule reposent uniquement sur les prédictions du modèle. Elles constituent une base solide pour le chapitre suivant, consacré à l'étude et à la préparation des données.

Chapitre 3 : Étude et préparation des données

3.1- Introduction

La qualité d'un modèle de Machine Learning dépend en grande partie des données sur lesquelles il est entraîné. Avant tout apprentissage, il est donc indispensable de choisir un **jeu de données adapté**, d'en comprendre la composition et de le préparer avec rigueur. Ce chapitre présente l'étude des datasets disponibles, la justification de notre choix, l'analyse quantitative et qualitative des données retenues, puis les différentes étapes de préparation : nettoyage, redimensionnement, normalisation et séparation en jeux d'entraînement, de validation et de test.

3.2- Étude des datasets disponibles

Plusieurs jeux de données publics ont été étudiés afin de trouver celui qui répond le mieux aux besoins du projet.

Tableau 9 *: Comparaison des datasets étudiés*

Dataset	Classes couvertes	Atouts	Limites
COCO 2017	Les 6	Photos réelles, une boîte annotée par objet	Fort déséquilibre, nombreux petits objets, recadrages à réaliser
CIFAR-10 et CIFAR-100	Les 6, en combinant les deux	Téléchargement simple, images déjà centrées	Images de 32 × 32 px seulement, peu de scènes routières
Pascal VOC 2012	5 sur 6	Annotations soignées, taille raisonnable	Absence de la classe camion
Jeux Kaggle « véhicules »	Variable	Prêts à l'emploi	Qualité et licences inégales, souvent sans personne ni vélo

3.3- Comparaison et justification du dataset retenu

Le descriptif du projet laisse le choix du jeu de données à l'équipe, en demandant seulement qu'il contienne des images d'objets routiers. Pour départager les options étudiées, nous avons défini cinq critères, directement liés aux besoins du projet :

- **Couverture des classes**
  Le dataset doit contenir les six classes retenues, sans devoir en abandonner une.
- **Réalisme des images**
  Les photos doivent montrer des objets dans des situations réelles et variées, proches de ce

que verrait un véhicule.

- **Résolution suffisante**
  Les objets doivent rester reconnaissables une fois recadrés, ce qui exclut les images trop petites.
- **Annotations par objet**
  Chaque objet doit être localisé par une boîte, afin de pouvoir le découper automatiquement.
- **Licence et documentation**
  Le dataset doit être documenté, cité dans la littérature et utilisable dans un cadre scolaire.

Tableau 10 *: Comparaison des datasets selon nos critères*

Critère	COCO 2017	CIFAR-10/100	Pascal VOC	Kaggle
Couverture des 6 classes	Oui	Oui, en combinant les deux	Non (5 sur 6)	Variable
Réalisme des images	Oui, scènes réelles variées	Faible, objets isolés	Oui	Variable
Résolution	Médiane de 640 × 480 px	32 × 32 px	Correcte	Variable
Annotations par objet	Oui, boîtes englobantes	Non, une étiquette par image	Oui	Variable
Documentation et licence	Article de référence, licences précisées	Documenté	Documenté	Inégales

Au regard de ces critères, nous avons retenu **COCO 2017**, sous réserve de la validation de notre encadrante. Il s'agit du seul dataset étudié qui remplit l'ensemble des critères : il couvre nos six classes avec de **vraies photos**, prises dans des conditions variées, et fournit des **annotations de référence** pour chaque objet. De plus, il n'est pas nécessaire de télécharger les 18 Go d'images, puisque seules les photos utiles sont récupérées grâce à leur adresse. CIFAR reste notre solution de secours, malgré sa résolution très faible.

3.3.1- Source et description du dataset retenu

Tableau 11 *: Fiche descriptive du dataset COCO 2017*

Élément	Description
Nom	COCO (*Common Objects in Context*), version 2017
Auteurs	Microsoft et plusieurs équipes universitaires, publication de référence de Lin et al. (2014) [1]
Source	Site officiel `https://cocodataset.org`, rubrique *Download*
Contenu	80 catégories d'objets du quotidien, dont nos six classes
Volume	118 287 images d'entraînement (`train2017`) et 5 000 images de validation (`val2017`)
Annotations	Fichiers JSON : catégorie, boîte englobante et indicateur de foule pour chaque objet
Licences	Annotations sous licence CC BY 4.0, images sous la licence Flickr choisie par chaque auteur
Utilisation dans le projet	`train2017` pour constituer nos jeux d'entraînement, de validation et de test, et `val2017` réservé aux tests sur de nouvelles images

Les licences imposent de citer COCO et de ne pas redistribuer les images. Celles-ci restent donc hors du dépôt Git, qui ne conserve que de petites vignettes dans les figures.

3.4- Acquisition des données à partir des annotations

3.4.1- Pourquoi récupérer les annotations plutôt que toutes les images

COCO se compose de deux parties distinctes : les **images**, d'une part, et les **annotations**, d'autre part, qui décrivent le contenu de chaque image dans un fichier texte au format JSON. Plutôt que de télécharger l'ensemble de la base, nous avons fait le choix de récupérer **uniquement les annotations**, puis de ne télécharger que les photos réellement utiles. Ce choix repose sur plusieurs raisons :

- **Un volume beaucoup plus faible**
  Les images de `train2017` représentent environ 18 Go, alors que l'archive des annotations ne pèse qu'environ 250 Mo. Or, nous n'avons besoin que de 10 672 photos sur 118 287, soit environ 9 % de la base.
- **Un stockage compatible avec Google Drive**
  Comme le montre le flux de développement (figure 2), nos données sont stockées dans un dossier Google Drive partagé. L'espace gratuit d'un compte Google étant limité à 15 Go, il ne permettrait pas d'accueillir les 18 Go d'images. Nos recadrages finaux n'occupent qu'environ 64 Mo.
- **Un gain de temps**
  Télécharger et décompresser 18 Go sur Colab à chaque session serait long, d'autant que le disque d'une session Colab est effacé à sa fermeture.

- **Une sélection possible avant tout téléchargement**
  Les annotations contiennent toutes les informations nécessaires pour choisir les objets, les filtrer et répartir les photos entre les jeux, sans ouvrir une seule image.

3.4.2- Comment les données sont récupérées

La récupération des données suit les étapes suivantes, réalisées dans les notebooks 01 et 02 :

1. **Téléchargement des annotations**
   L'archive `annotations_trainval2017.zip` est téléchargée depuis le site officiel de COCO, puis décompressée. Le fichier `instances_train2017.json` décrit 118 287 images, 860 001 objets et 80 catégories.

2. **Exploration à partir des annotations**
   Les effectifs par classe, la taille des objets et le déséquilibre présentés dans ce chapitre sont calculés directement à partir de ce fichier, sans télécharger d'image.

3. **Sélection des objets**
   Nous conservons les objets de nos six classes, en écartant les foules et les objets de moins de 32 px de côté, puis nous tirons au plus 2 500 objets par classe.

4. **Répartition des photos**
   Les objets sont répartis entre les jeux d'entraînement, de validation et de test selon l'identifiant de leur photo, toujours à partir des annotations.

5. **Téléchargement ciblé**
   Chaque photo sélectionnée est téléchargée une seule fois grâce à son adresse (`coco_url`). Tous ses objets sont alors recadrés à l'aide de leur boîte, et seuls les recadrages de 64 × 64 px sont conservés. La photo complète n'est pas stockée.

6. **Stockage sur Google Drive**
   Les annotations, la liste des objets retenus, le fichier de découpage et les recadrages sont enregistrés dans le dossier Drive partagé de l'équipe. Les recadrages sont également regroupés dans une archive, décompressée sur le disque de Colab au début de l'entraînement, ce qui évite de lire des milliers de petits fichiers depuis Drive.

Tableau 12 *: Volume des données stockées sur Google Drive*

Élément	Volume	Rôle
Archive des annotations	Environ 250 Mo	Source de toutes les informations sur les objets
Fichier `instances_train2017.json`	Environ 470 Mo	Annotations décompressées, lues par les notebooks
Recadrages de 64 × 64 px	14 990 images, environ 64 Mo	Données d'entraînement, de validation et de test
Fichiers `objets_coco.csv` et `splits.csv`	Quelques Mo	Liste des objets et découpage commun à l'équipe
Images complètes de COCO	Non stockées	Téléchargées une à une, puis supprimées après recadrage

3.4.3- Exemple d'annotation

Pour illustrer ce fonctionnement, voici une annotation réelle extraite du fichier `instances_train2017.json`. Elle décrit une voiture présente sur la photo `000000384827.jpg`. Les informations utiles sont réparties entre trois parties du fichier : la description de l'image, la description de l'objet et la catégorie.

```
"images": [ { "id": 384827, "file_name": "000000384827.jpg", "width": 640, "height": 480,
            "coco_url": "http://images.cocodataset.org/train2017/000000384827.jpg",
            "license": 2, "date_captured": "2013-11-17 07:28:42" } ]

"annotations": [ { "id": 1193435, "image_id": 384827, "category_id": 3,
                "bbox": [177.98, 287.87, 81.98, 80.9], "area": 2355.63, "iscrowd": 0,
                "segmentation": [[177.98, 287.87, 231.91, 291.11, ...]] } ]

"categories": [ { "id": 3, "name": "car", "supercategory": "vehicle" } ]
```

Figure 8 *: Extrait réel du fichier d'annotations de COCO `train2017` (segmentation raccourcie)*

Tableau 13 *: Champs de l'annotation utilisés dans le projet*

Champ	Valeur de l'exemple	Utilisation dans le projet
`category_id`	3, soit « car »	Attribue la classe « voiture » sans étiquetage manuel
`bbox`	[177,98 ; 287,87 ; 81,98 ; 80,9]	Position (x, y) du coin supérieur gauche, largeur et hauteur de la boîte, utilisées pour le recadrage
`iscrowd`	0	Indique un objet isolé : l'objet est conservé
Plus petit côté de `bbox`	80,9 px	Supérieur à 32 px : l'objet passe le filtre de taille
`image_id`	384827	Rattache l'objet à sa photo, ce qui permet le découpage par photo
`coco_url`	Adresse de la photo	Permet de télécharger uniquement cette photo
`width`, `height`	640 et 480 px	Permettent de gérer les recadrages qui dépassent le bord de la photo
`license`	2, soit CC BY-NC 2.0	Permet de connaître les conditions d'utilisation de la photo
`segmentation`	Contour détaillé de l'objet	Non utilisé : la boîte englobante suffit à notre classification

À partir de cette annotation, le recadrage est obtenu automatiquement : un carré de 98 px de côté, soit le plus grand côté de la boîte augmenté d'une marge de 10 % de chaque côté, est découpé autour du centre de la voiture, puis réduit à 64 × 64 px (figure 9). Il faut noter que cette même photo contient quatre voitures annotées. Grâce à leur `image_id` commun, elles sont toutes placées dans le même jeu de données.

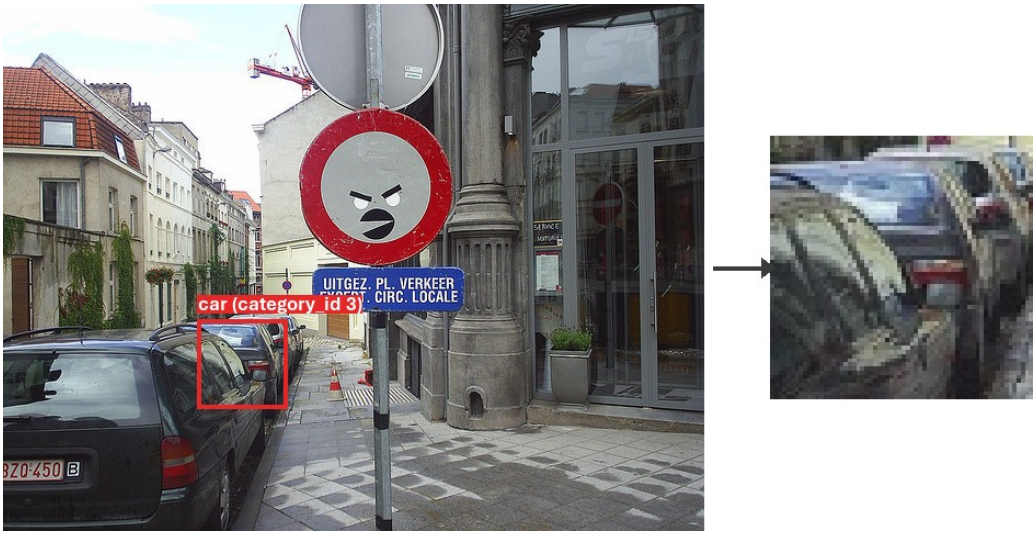


Figure 9 *: De l'annotation au recadrage : boîte de l'annotation 1193435 sur la photo d'origine, puis recadrage de 64 × 64 px (photo COCO 000000384827, licence CC BY-NC 2.0)*

3.4.4- Bénéfices de l'utilisation des annotations

L'utilisation des annotations présentes dans COCO apporte plusieurs bénéfices majeurs au projet :

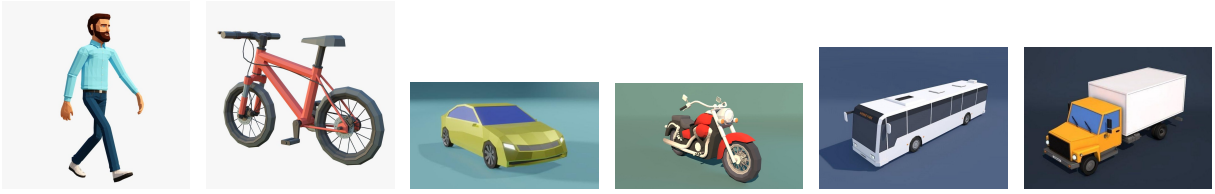
- **Aucun étiquetage manuel**
  Chaque objet possède déjà sa catégorie, vérifiée par les équipes de COCO. Étiqueter et encadrer nous-mêmes 15 000 objets n'aurait pas été réalisable dans les délais du projet.
- **Un découpage automatique**
  Les boîtes englobantes permettent d'extraire chaque objet de sa photo sans intervention humaine.
- **Un nettoyage simple et traçable**
  Les champs `iscrowd` et `bbox` suffisent à écarter les foules et les objets trop petits, et chaque objet conservé reste identifié par son numéro d'annotation.
- **Une évaluation fiable**
  Le champ `image_id` permet de regrouper les objets par photo et de garantir qu'aucune photo n'apparaît dans deux jeux différents.
- **Une exploration rapide**
  Les statistiques sur les classes et la taille des objets sont obtenues en quelques secondes, avant tout téléchargement d'image.
- **Un respect des licences**
  Chaque image est liée à sa licence, ce qui permet de connaître ses conditions d'utilisation.

3.5- Présentation des classes

Les six classes retenues correspondent aux usagers de la route que le véhicule doit savoir reconnaître :

- **Personne (`person`)**
  Il s'agit de l'usager le plus vulnérable. Une erreur sur cette classe est donc la plus grave pour un véhicule.
- **Vélo (`bicycle`)**
  Usager lent et fragile, souvent accompagné d'une personne. Il faut noter que COCO annote le vélo et non le cycliste.
- **Voiture (`car`)**
  Usager le plus fréquent, souvent à l'arrêt sur la chaussée.
- **Moto (`motorcycle`)**
  Véhicule léger et rapide, visuellement proche du vélo.
- **Bus (`bus`)**
  Grand véhicule qui masque une partie de la visibilité.
- **Camion (`truck`)**
  Grand véhicule. COCO range également dans cette classe des pick-ups et des camionnettes.

L'ordre des classes (personne, vélo, voiture, moto, bus, camion) est fixé une fois pour toutes, car il correspond aux sorties du modèle et à ce qu'attend le simulateur.



Personne *Vélo* *Voiture* *Moto* *Bus* *Camion*

Figure 10 *: Illustration des six classes par des modèles 3D (le modèle apprend sur des photos réelles de COCO)*

3.6- Analyse quantitative des données

L'ensemble `train2017` de COCO compte **118 287 images**, dont **70 082** contiennent au moins un objet de nos classes. La résolution médiane de ces images est de 640 × 480 px.

Tableau 14 *: Nombre d'objets disponibles par classe dans COCO `train2017`*

Classe	Objets annotés	Hors foules, plus petit côté d'au moins 32 px
Personne	262 465	150 185
Voiture	43 867	16 799
Camion	9 973	7 071
Moto	8 725	6 742
Vélo	7 113	4 505
Bus	6 069	5 194

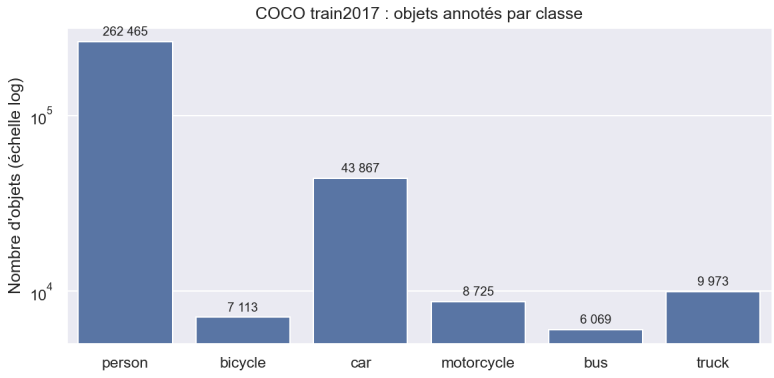


Figure 11 *: Répartition des objets par classe avant filtrage*

Ces chiffres mettent en évidence un **fort déséquilibre** entre les classes : on compte 43,2 personnes pour un bus. Sans précaution, un modèle pourrait obtenir une bonne accuracy en répondant presque toujours « personne ». C'est pourquoi nous limitons le nombre d'objets par classe et évaluons les modèles avec le F1 macro.

3.7- Analyse qualitative des images

L'observation d'exemples tirés au hasard a permis de relever plusieurs difficultés :

- des objets **coupés** par le bord de la photo ou **cachés** par un autre objet ;
- des images **floues** ou très petites, en particulier pour les voitures éloignées ;
- **plusieurs objets** dans un même recadrage, comme un motard sur sa moto ;
- une classe « camion » **hétérogène**, qui contient aussi des pick-ups et des camionnettes, proches d'une voiture ;
- des photos prises par des particuliers et non depuis un véhicule, ce qui constitue un biais à garder en tête pour un usage routier.

3.8- Nettoyage des données

Afin de ne conserver que des exemples exploitables, plusieurs règles de nettoyage ont été appliquées :

- **Exclusion des foules**
  Une boîte marquée `iscrowd` entoure un groupe de personnes et non un seul usager.
- **Exclusion des petits objets**
  En dessous de 32 px de côté, l'objet devient trop flou une fois agrandi. À ce seuil, la classe la plus rare conserve 4 505 objets, contre seulement 2 693 avec un seuil de 64 px.
- **Plafond de 2 500 objets par classe**
  Ce plafond permet d'équilibrer les classes tout en gardant un entraînement réalisable sans GPU.
- **Contrôle des téléchargements**
  Une photo inaccessible est notée sans interrompre la préparation. Sur les 15 000 objets prévus, 14 990 ont été récupérés, 8 téléchargements ayant expiré.

3.9- Redimensionnement et normalisation

Chaque objet est découpé dans un **carré** centré sur sa boîte, avec une marge de 10 %, puis réduit à **64 × 64 px**. Le recadrage carré évite de déformer les silhouettes, une personne étant haute et fine alors qu'un bus est large. Lorsque le carré dépasse les bords de la photo, les pixels manquants sont complétés par **reflet** plutôt que par une bande noire, que le modèle pourrait apprendre à tort.

Chaque canal de couleur est ensuite **normalisé** à l'aide de la moyenne et de l'écart-type calculés sur le jeu d'entraînement, soit [0,4057 ; 0,3894 ; 0,3748] pour la moyenne et [0,2609 ; 0,2554 ; 0,2571] pour l'écart-type. Le simulateur applique exactement les mêmes valeurs.

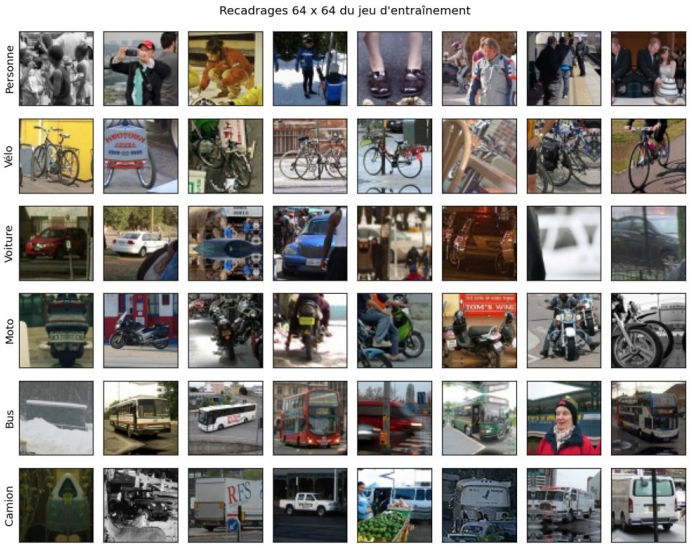


Figure 12 *: Recadrages de 64 × 64 px du jeu d'entraînement, une ligne par classe*

3.10- Séparation entraînement, validation et test

Les données ont été réparties en trois jeux : **70 %** pour l'entraînement, **15 %** pour la validation et **15 %** pour le test.

Tableau 15 *: Répartition des images par classe et par jeu*

Classe	Entraînement	Validation	Test
Personne	1 750	375	375
Vélo	1 748	373	375
Voiture	1 750	375	374
Moto	1 750	374	375
Bus	1 748	374	375
Camion	1 749	375	375

Le découpage est **stratifié**, afin de conserver les mêmes proportions de classes dans chaque jeu. Il est également réalisé **par photo d'origine**, car deux objets d'une même photo partagent le même éclairage et le même décor. Les répartir entre l'entraînement et le test fausserait donc le score. Un contrôle automatique vérifie qu'aucune des 10 672 photos n'apparaît dans deux jeux, et le découpage est enregistré dans un fichier commun afin que toute l'équipe utilise exactement les mêmes données.

3.11- Conclusion

Ce chapitre a permis de justifier le choix de **COCO 2017** et de décrire précisément les données retenues. L'analyse a mis en évidence un fort déséquilibre entre les classes ainsi que plusieurs difficultés liées à la qualité des images. Les étapes de nettoyage, de recadrage, de normalisation et de découpage par photo garantissent un jeu de données équilibré et une évaluation fiable. Ces

données préparées serviront de base au modèle de classification, dont l'intégration dans l'architecture globale est détaillée dans le chapitre suivant.

Chapitre 4 : Architecture technique et conception des modules

4.1- Introduction

Après avoir défini les besoins du système et préparé les données, il est désormais essentiel d'aborder l'**aspect technique** du projet. Ce chapitre a pour objectif de présenter l'architecture globale d'AutoMaze AI, en détaillant le rôle de chaque module ainsi que leurs interactions. Une attention particulière sera accordée à la **séparation entre la vérité terrain du simulateur et la perception du modèle**, qui garantit que les décisions du véhicule reflètent réellement la qualité de notre classifieur.

4.2- Architecture globale d'AutoMaze AI

L'architecture du projet repose sur un principe simple. Le simulateur connaît la position de chaque objet présent dans son environnement, tandis que le **modèle de Machine Learning** analyse l'image fournie par la caméra virtuelle et en détermine la nature. Le système associe ensuite la classe prédite à la position connue de l'objet. Le **planificateur** calcule le chemin vers la destination et le recalcule lorsqu'un obstacle rend le trajet impraticable. Enfin, le **contrôleur** applique les décisions au véhicule virtuel : continuer, ralentir, s'arrêter ou changer de direction.

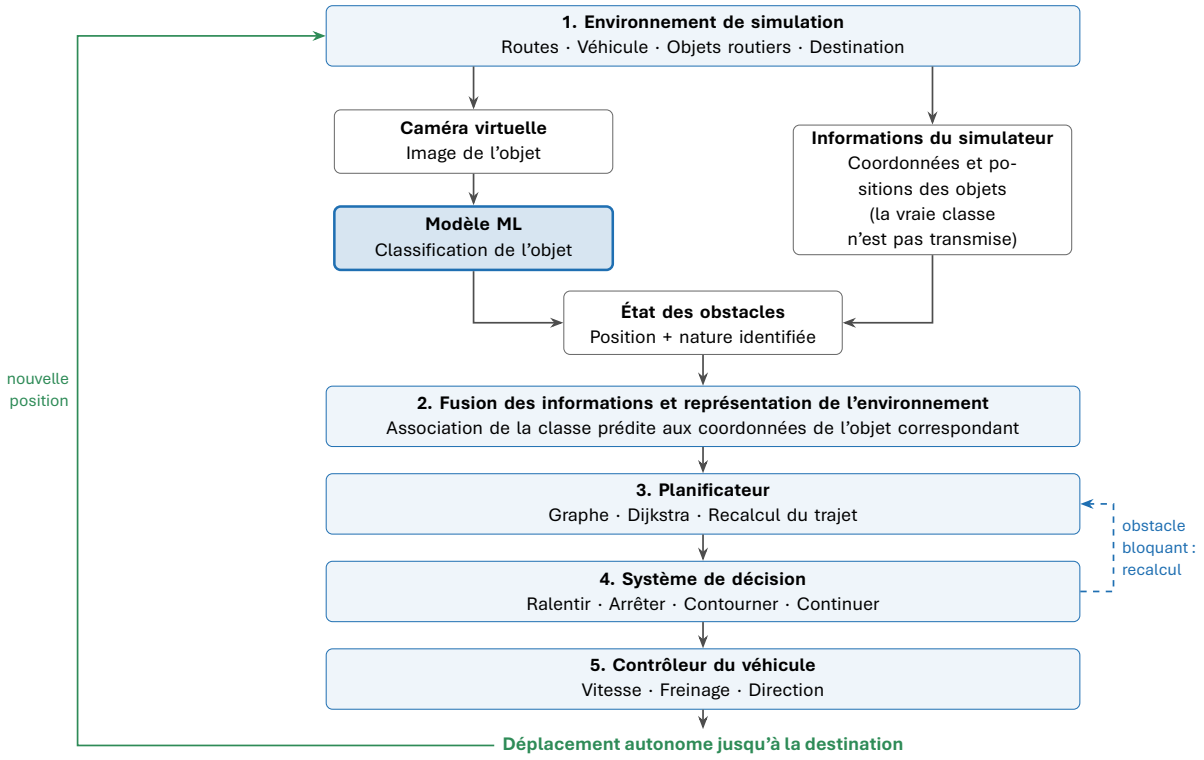


Figure 13 *: Architecture fonctionnelle globale d'AutoMaze AI*

4.3- Environnement de simulation

Notre environnement de simulation reproduit une partie réelle de **Casablanca**, du Twin Center (Maârif) à la place Mohammed V. Le réseau routier provient d'**OpenStreetMap** et respecte les sens uniques. Le véhicule circule à 8 m/s, soit environ 29 km/h, et le temps avance par pas d'une seconde. Chaque épisode place six obstacles sur l'itinéraire, et trois vues sont proposées : une carte en 2D, une vue conducteur et une vue en 3D.

Les simulateurs de conduite professionnels offrent des environnements 3D très détaillés, comme l'illustre la figure 14. Notre simulateur reste volontairement plus simple, puisqu'il sert uniquement à démontrer l'utilisation du modèle.



Figure 14 *: Exemples d'environnements virtuels de conduite ayant servi d'inspiration (sources externes)*

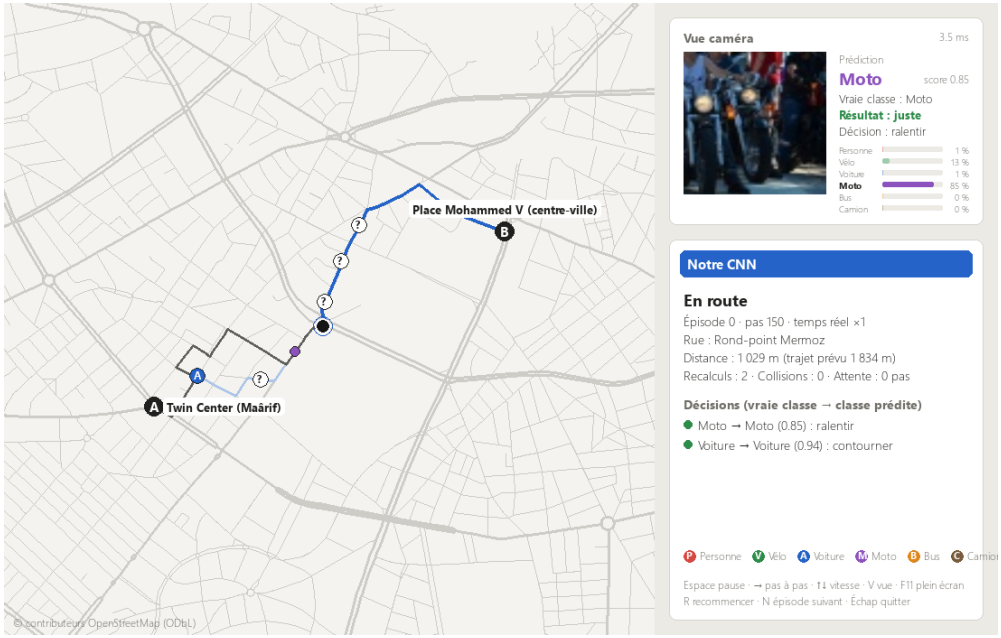


Figure 15 *: Notre simulateur en vue carte : itinéraire, obstacles et dernière prédiction (carte © contributeurs OpenStreetMap)*

4.4- Acquisition des images par caméra virtuelle

Lorsqu'un obstacle se trouve sur l'itinéraire à moins de 35 m, la **caméra virtuelle** transmet son image au modèle.

Ce module repose sur un choix de conception important. Si la caméra filmait les objets 3D du simulateur, le modèle recevrait des images de synthèse très différentes des photos sur lesquelles il a appris, et ses performances chuteraient. La caméra fournit donc une **vraie photo** de la classe de l'obstacle, tirée du jeu de test, que le modèle n'a jamais vue. Les modèles 3D servent uniquement à l'affichage. De plus, une même graine aléatoire donne les mêmes images à tous les classifieurs comparés, tandis que les images changent d'un épisode à l'autre.

4.4.1- Association d'une image à chaque objet

L'association entre un objet du simulateur et l'image transmise au modèle se déroule en quatre étapes :

1. **Création de l'obstacle**
   Au début de chaque épisode, le simulateur place six obstacles sur l'itinéraire. Chacun reçoit une position (un tronçon de rue et une distance sur ce tronçon) ainsi qu'une vraie classe, tirée au hasard parmi les six.

2. **Choix de l'image**
   Lorsque le véhicule arrive à moins de 35 m de l'obstacle, la caméra virtuelle choisit une image dans le dossier du jeu de test correspondant à sa vraie classe (`data/crops/test/<classe>/`). Si l'obstacle est un camion, la caméra « voit » donc la photo d'un vrai camion que le modèle n'a jamais rencontrée.

3. **Tirage reproductible**
   Le choix de l'image dépend de la graine de l'expérience, de la position de l'obstacle et du numéro de l'épisode. Ainsi, un même obstacle reçoit toujours la même image, quel que soit le classifieur testé, ce qui garantit une comparaison équitable. L'image est ensuite conservée en mémoire, afin que le véhicule voie la même photo s'il repasse devant l'obstacle.

4. **Transmission au modèle**
   L'image est transmise au classifieur actif, sans aucune indication sur sa vraie classe. La prédiction obtenue est alors rattachée à l'obstacle d'origine (section 4.6).

La vraie classe sert donc uniquement à choisir ce que la caméra « voit », comme dans la réalité, où un camion présent sur la route apparaît sur l'image de la caméra. Elle n'intervient jamais dans la décision du véhicule, qui repose seulement sur la prédiction du modèle. Si aucune image n'est disponible pour une classe, par exemple avant la préparation des données, le simulateur génère une image de remplacement.

4.5- Module de classification par Machine Learning

Le module de classification reçoit l'image RGB d'un objet, quelle que soit sa taille, et applique lui-même le prétraitement : mise au carré, réduction à 64 × 64 px et normalisation. Il renvoie ensuite :

- la **classe prédite**, parmi les six classes ;
- un **score de confiance**, correspondant au plus grand des six scores ;

- les **six scores**, dont la somme vaut 1 ;
- le **temps de calcul** réellement mesuré.

L'affichage prend par exemple la forme « Image → Voiture (92 %) ». Ce pourcentage est un exemple d'affichage et non une performance mesurée. Il s'agit d'un score et non d'une probabilité calibrée, ce qui signifie que le modèle n'a pas forcément raison 92 fois sur 100.

Lorsque le score est inférieur à un certain seuil, la classe devient « Inconnu » et le véhicule effectue un arrêt prudent. Ce **seuil** sera choisi sur le jeu de validation, puis enregistré avec le modèle.

Enfin, trois classifieurs respectent la même interface et sont donc interchangeables : un mode « simulation », qui permet de tester la logique sans modèle, notre propre modèle, et un éventuel modèle de référence préentraîné, utilisé uniquement comme point de comparaison si l'encadrante l'accepte.

4.6- Module de localisation et association des objets

Le simulateur décrit chaque obstacle par un identifiant et une position sur la carte, comme le montre l'exemple suivant.

Tableau 16 *: Exemple d'association entre position et classe prédite*

Identifiant	Position	Classe prédite	Vraie classe (cachée)
OBJ-01	Tronçon A → B, à 45 m de A	Voiture	Voiture
OBJ-02	Tronçon C → D, à 12 m de C	Personne	Personne
OBJ-03	Tronçon E → F, à 30 m de E	Voiture	Camion

L'**association** entre une prédiction et un objet est directe : chaque image est produite pour un obstacle précis, et la prédiction est donc rattachée à l'objet dont elle provient. Ce choix, simple et sûr, est possible car chaque image ne contient qu'un seul objet.

Il est essentiel de bien séparer deux types d'informations :

- **La vérité terrain**
  Elle regroupe la vraie classe et la position de l'objet. Elle sert à construire les scénarios, à faire agir l'obstacle selon sa vraie nature (une personne finit de traverser, un camion reste immobile) et à mesurer les erreurs.
- **La perception**
  Elle correspond à la classe prédite par notre modèle. C'est la seule information transmise au système de décision. Une erreur du modèle a donc de vraies conséquences, comme dans le cas de l'objet OBJ-03, qui peut mener à une collision.

4.7- Représentation de l'environnement et des obstacles

L'environnement est représenté sous forme de **graphe orienté**, dont les nœuds sont les intersections et les arêtes les tronçons de rue. Chaque tronçon possède un coût égal à sa longueur en mètres, éventuellement multipliée par une pénalité. Un obstacle peut modifier ce graphe de trois manières :

- en **bloquant** un tronçon, définitivement ou pendant quelques pas ;
- en **pénalisant** un tronçon, c'est-à-dire en multipliant son coût ;
- en pénalisant également les **tronçons voisins**, dans le cas d'un gros véhicule.

4.8- Algorithme de planification Dijkstra

L'algorithme de **Dijkstra**, implémenté directement dans le projet plutôt qu'emprunté à une bibliothèque, calcule le chemin de coût minimal entre la position actuelle du véhicule et sa destination. Après chaque modification du graphe, il est relancé depuis la position du véhicule. Si l'obstacle se trouve sur le tronçon actuel, le véhicule peut faire demi-tour, et les sens uniques sont toujours respectés.

Il faut toutefois noter que le chemin le plus court n'est pas toujours le plus rapide ni le plus sûr. Nous utilisons donc les distances comme coûts de départ, puis nous ajoutons des pénalités pour les passages moins favorables, par exemple une rue où circule un vélo.

4.8.1- Justification du choix de Dijkstra

Le choix de Dijkstra répond d'abord à un **objectif pédagogique** : il s'agit de mettre en pratique, dans un cas concret, l'un des algorithmes de graphes étudiés en première année. La planification n'étant pas le cœur scientifique du projet, nous avons privilégié un algorithme que l'équipe maîtrise et peut expliquer en détail, plutôt qu'une méthode plus avancée.

Ce choix reste par ailleurs **adapté au problème**, pour plusieurs raisons :

- **Des coûts toujours positifs**
  Les coûts sont des longueurs en mètres, éventuellement multipliées par une pénalité supérieure à 1. Ils sont donc toujours positifs, ce qui est la condition pour que Dijkstra trouve le plus court chemin exact.
- **Un graphe de taille modeste**
  La zone simulée compte 1 387 intersections et 2 907 tronçons de rue. Avec une file de priorité, Dijkstra calcule l'itinéraire de 1 834 m entre le Twin Center et la place Mohammed V en 5 ms environ sur un processeur ordinaire, ce qui permet de le relancer après chaque obstacle.
- **Un résultat exact et facile à vérifier**
  L'algorithme ne repose sur aucune approximation, ce qui facilite l'écriture de tests automatiques sur de petits graphes dont la solution est connue.

Tableau 17 *: Comparaison des algorithmes de plus court chemin envisagés*

Algorithme	Intérêt	Limite pour notre projet
Dijkstra (retenu)	Chemin le plus court exact avec des coûts positifs, algorithme étudié en première année	Explore plus de nœuds que A*, ce qui reste sans conséquence sur un graphe de cette taille
Parcours en largeur (BFS)	Très simple	Ne tient pas compte des longueurs des rues, seulement du nombre de tronçons
A*	Plus rapide sur de très grands graphes grâce à une estimation de la distance restante	Gain inutile ici, et une heuristique supplémentaire à justifier
Bellman-Ford	Accepte des coûts négatifs	Plus lent, alors que nos coûts sont toujours positifs

4.9- Système de décision

Le système de décision traduit les informations de perception et de planification en actions. Une décision n'est déclenchée que pour un obstacle situé sur l'itinéraire, devant le véhicule et à moins de 35 m.

Tableau 18 *: Règles de décision selon la situation rencontrée*

Situation	Décision	Effet
Aucun obstacle sur la trajectoire	Continuer	Le véhicule suit son itinéraire
Personne	S'arrêter et attendre	Attente de 5 pas, puis reprise du même trajet
Vélo ou moto	Ralentir	Tronçon 3 fois plus coûteux, vitesse divisée par 3 s'il reste le meilleur chemin
Voiture	Contourner	Tronçon bloqué, puis nouvel itinéraire
Bus ou camion	Contourner largement	Tronçon bloqué, rues voisines 10 fois plus coûteuses, puis nouvel itinéraire
Score trop faible (« Inconnu »)	Arrêt prudent	Attente de 2 pas, tronçon bloqué, puis nouvel itinéraire
Aucun chemin possible	Rester arrêté	Nouvel essai à chaque pas, épisode « bloqué » après 30 pas
Voie dégagée (fin de l'attente)	Reprendre	Le véhicule repart sur son itinéraire

Dans cette première version, l'action dépend uniquement de la classe prédite, la distance et la position de l'objet servant seulement à déclencher la décision. Une règle plus réaliste devrait également tenir compte de la distance, de la trajectoire et de la vitesse relative. En effet, un piéton sur le trottoir ne demande pas le même arrêt qu'un piéton sur la chaussée. Ce raffinement pourra être ajouté si le temps le permet.

4.10- Contrôle du véhicule virtuel

Le contrôleur reçoit les actions décidées et commande le véhicule virtuel. Il reste volontairement simplifié et ne modélise ni les forces ni l'adhérence. À chaque pas, il applique l'une des commandes suivantes :

- **avancer** de 8 m le long de l'itinéraire ;
- **ralentir**, en divisant la vitesse par 3 sur un tronçon pénalisé ;
- **s'arrêter** pendant le nombre de pas décidé ;
- **changer de direction** en suivant le nouvel itinéraire, ou faire demi-tour si nécessaire ;
- **reprendre** le déplacement lorsque la voie est libre.

Une collision est comptée lorsque le véhicule atteint un obstacle encore présent. Cette situation peut se produire, par exemple, si un camion immobile a été confondu avec un vélo : le véhicule ralentit alors au lieu de contourner.

4.11- Communication et intégration des modules

La communication entre les modules repose sur quelques points d'échange bien définis, qui limitent les risques d'incompatibilité.

Tableau 19 *: Points d'échange entre les modules*

Point d'échange	Rôle
`simulator/contracts.py`	Seul fichier partagé entre les modules, il définit la prédiction, l'obstacle, les actions et l'interface commune des classifieurs
`models/metadata.json`	Décrit le modèle : ordre des classes, taille d'entrée, ordre des couleurs, normalisation et seuil
Poids du modèle	Chargés par le simulateur, qui ne réentraîne jamais le modèle
Journal des rencontres	Une ligne par obstacle (vraie classe, classe prédite, score, action, temps), utilisée pour mesurer les erreurs

Au chargement du modèle, le simulateur vérifie que l'ordre des classes est correct, car une inversion transformerait par exemple un bus en personne sans aucun message d'erreur. Un test de concordance compare également les sorties du simulateur et celles du notebook sur les mêmes images, afin de garantir que le prétraitement est identique.

4.12- Conclusion

Ce chapitre a permis de présenter l'**architecture technique d'AutoMaze AI** et le fonctionnement détaillé de chacun de ses modules. L'approche retenue garantit une séparation claire entre la vérité terrain du simulateur et la perception du modèle, ce qui permet de mesurer réellement l'apport de la classification. Les choix effectués pour la caméra virtuelle, l'association des objets, la

planification et la décision assurent la cohérence de l'ensemble. Le chapitre suivant présente la méthodologie expérimentale qui permettra d'entraîner et d'évaluer les classifieurs.

Chapitre 5 : Méthodologie expérimentale et évaluation

5.1- Introduction

Une fois les données préparées et l'architecture définie, il est nécessaire de fixer une **méthode d'expérimentation rigoureuse** pour entraîner, comparer et sélectionner les modèles. Ce chapitre présente l'extraction des caractéristiques visuelles, les algorithmes de classification étudiés, le protocole expérimental et les métriques d'évaluation. Il décrit également la méthode de comparaison, les critères de sélection du modèle final ainsi que le protocole de validation dans le simulateur.

5.2- Extraction des caractéristiques

Les algorithmes SVM, Random Forest et k-NN ne peuvent pas exploiter directement une image. Il faut d'abord la résumer en une liste de nombres, appelée **descripteur**. Nous utilisons le descripteur **HOG** (*Histogram of Oriented Gradients*), qui décrit l'orientation des contours présents dans l'image. Son calcul se déroule en trois étapes :

1. l'image de 64 × 64 px est convertie en niveaux de gris ;
2. elle est découpée en cellules de 8 × 8 px, et chaque cellule compte ses contours selon 9 orientations ;
3. les cellules sont regroupées en blocs de 2 × 2, normalisés afin de compenser les différences d'éclairage.

Un bloc de 16 px, décalé de 8 px à chaque fois, occupe 7 positions par côté, ce qui donne $7 \times 7 = 49$ blocs de 4×9 valeurs, soit $49 \times 4 \times 9 = 1\,764$ nombres par image. Nous testerons également des cellules de 4 px, plus fines mais beaucoup plus nombreuses. Il faut noter que HOG ne tient pas compte de la couleur, ce qui pourrait pénaliser les classes de formes proches, comme le bus et le camion.

5.3- Présentation des algorithmes SVM, Random Forest et k-NN

Trois algorithmes de classification, prévus par le sujet, sont comparés dans ce projet :

- **SVM (Support Vector Machine)**
  Le SVM recherche la frontière qui sépare les classes avec la plus grande marge possible. Avec un noyau RBF, cette frontière peut être courbe. Il est efficace en grande dimension, mais devient lent à entraîner lorsque les exemples sont nombreux. Nous ferons varier le type de noyau (linéaire ou RBF) ainsi que le paramètre C.

- **Random Forest**
  Le Random Forest fait voter plusieurs centaines d'arbres de décision, chacun entraîné sur un tirage différent des données. Il est rapide et robuste, mais chaque question d'un arbre ne porte que sur une seule valeur à la fois. Nous ferons varier le nombre d'arbres et leur profondeur.

- **k-NN (k plus proches voisins)**
  Le k-NN attribue à une image la classe majoritaire parmi les k images d'entraînement les plus proches. Il est simple et ne nécessite pas de véritable entraînement, mais les distances perdent leur sens en grande dimension. Nous ferons donc varier k ainsi que le nombre de dimensions conservées après réduction par PCA (de 50 à 200).

Le SVM et le k-NN sont précédés d'une mise à l'échelle des valeurs, car ils y sont sensibles, ce qui n'est pas nécessaire pour le Random Forest.

En complément des trois algorithmes du sujet, et sous réserve de l'accord de notre encadrante, nous proposons deux **extensions** : **XGBoost**, une autre famille d'arbres construits successivement, et un petit **réseau convolutif (CNN)** entraîné de zéro. Ces extensions seront comparées de la même manière, sans remplacer les trois modèles demandés.

5.4- Protocole expérimental

Le protocole expérimental repose sur cinq principes :

1. **Un même point de départ** : tous les modèles utilisent les mêmes recadrages, le même découpage et les mêmes descripteurs.
2. **Des réglages sur la validation** : chaque combinaison de paramètres est entraînée sur le jeu d'entraînement, puis mesurée sur le jeu de validation.
3. **Un seul changement à la fois** : chaque essai est consigné avec ses réglages, son score et son temps d'entraînement.
4. **Un test ouvert une seule fois** : les modèles retenus sont mesurés sur le jeu de test à la toute fin, et plus aucun réglage n'est permis ensuite.
5. **La reproductibilité** : les graines aléatoires sont fixées, et les notebooks s'exécutent de bout en bout.

Le jeu de test reste fermé jusqu'à la fin, car un modèle réglé en observant ce jeu obtiendrait un score trop optimiste, à la manière d'un élève qui connaîtrait à l'avance les questions de l'examen.

5.5- Métriques d'évaluation

Plusieurs métriques sont utilisées pour évaluer les modèles :

Tableau 20 : *Métriques d'évaluation retenues*

Métrique	Ce qu'elle mesure
Accuracy	Part de bonnes réponses sur l'ensemble des images
Précision (par classe)	Lorsque le modèle répond « camion », part de vrais camions
Rappel (par classe)	Parmi les vrais camions, part que le modèle retrouve
F1 (par classe)	Moyenne harmonique de la précision et du rappel : $F1 = \frac{2 \times \text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}$
F1 macro	Moyenne des F1 des six classes, chacune ayant le même poids. Il s'agit de notre **métrique principale**
Rappel « personne »	Part des personnes reconnues, l'erreur la plus grave pour un véhicule
Matrice de confusion	Croisement des vraies classes et des classes prédites, pour identifier les confusions
Temps d'inférence	Temps de calcul d'une prédiction sur un processeur ordinaire

5.6- Méthode de comparaison des modèles

La comparaison des modèles s'appuie sur les éléments suivants :

- **Des données identiques**
  Les modèles sont comparés sur les mêmes images, dans le même ordre.
- **Des scores planchers**
  Un modèle qui répond toujours la même classe, ou qui tire au hasard selon les fréquences, fournit la valeur minimale à dépasser.
- **L'analyse des erreurs**
  Au-delà du score global, nous comparons les matrices de confusion et observons des images mal classées tirées au hasard.
- **La prise en compte de l'incertitude**
  Avec environ 2 250 images de test, une accuracy n'est connue qu'à environ $\pm$ 2 points près. Un écart plus faible ne permet donc pas de départager deux modèles.

5.7- Critères de sélection du modèle final

Le modèle final sera choisi selon les critères suivants, par ordre de priorité :

1. le meilleur **F1 macro sur la validation** ;
2. à score proche, le meilleur **rappel « personne »** ;
3. un **temps d'inférence** compatible avec le simulateur, de l'ordre de quelques millisecondes ;
4. enfin, le modèle le plus simple et le plus stable d'un essai à l'autre.

Le modèle retenu sera exporté avec ses métadonnées. Son seuil « Inconnu » sera fixé sur la valida-

tion, en retenant le seuil le plus élevé qui laisse au moins 90 % des images classées.

5.8- Protocole de validation dans le simulateur

Pour la validation dans le simulateur, chaque classifieur parcourt exactement les mêmes épisodes : même carte, même itinéraire, mêmes obstacles et mêmes images. Le protocole prévoit **50 épisodes**, avec une graine fixée.

Tableau 21 *: Mesures relevées dans le simulateur*

Mesure	Définition
Épisodes arrivés	Le véhicule a atteint sa destination
Collisions	Le véhicule a atteint un obstacle encore présent
Obstacles bien classés	Classe prédite identique à la vraie classe
Fautes graves	Personne classée autrement que « personne », « Inconnu » compris
Décisions inadaptées	Action différente de celle que demandait la vraie classe
Détour et recalculs	Distance parcourue rapportée au trajet prévu, et nombre de nouveaux calculs
Temps par décision	Temps moyen d'une prédiction, mesuré sur processeur

Il faut préciser que le mode « simulation » n'est pas un modèle, puisque son taux d'erreur est réglé manuellement. Il permet uniquement de vérifier la logique du simulateur, et ses résultats ne peuvent pas être comparés à ceux d'un véritable classifieur.

5.9- Conclusion

Ce chapitre a permis de définir une **méthodologie expérimentale claire et reproductible**. L'extraction des descripteurs HOG, la comparaison des trois classifieurs du sujet sur des données identiques et l'utilisation du F1 macro comme métrique principale garantissent une évaluation équitable. Le protocole retenu, qui réserve le jeu de test à l'évaluation finale, assure la fiabilité des résultats. Le dernier chapitre s'appuie sur ces éléments pour étudier la faisabilité du projet et préparer la décision de Go/No-Go.

Chapitre 6 : Étude de faisabilité et validation Go/No-Go

6.1- Introduction

Ce dernier chapitre a pour objectif d'évaluer la **faisabilité du projet** et de préparer la décision de Go/No-Go. Il présente successivement la faisabilité technique, matérielle et logicielle, l'identification des risques et des parades associées, ainsi que les preuves de faisabilité déjà disponibles. Il expose enfin le planning de réalisation et les critères sur lesquels repose la décision.

6.2- Faisabilité technique

Chaque brique technique du projet s'appuie sur des méthodes reconnues et des bibliothèques largement utilisées :

- **Données**
  COCO fournit des boîtes annotées pour nos six classes, ce qui permet de découper les objets automatiquement.
- **Classification**
  HOG, SVM, Random Forest et k-NN sont disponibles dans les bibliothèques OpenCV et scikit-learn.
- **Planification**
  L'algorithme de Dijkstra s'applique directement à un graphe routier, et OpenStreetMap fournit la carte de Casablanca.
- **Intégration**
  Le simulateur et le modèle ne communiquent que par un fichier de métadonnées et une interface commune, ce qui limite les risques d'incompatibilité.

La principale difficulté technique se situe au niveau des données : classes déséquilibrées, petits objets et étiquettes parfois discutables. Le chapitre 3 a présenté les solutions retenues pour y répondre.

6.3- Faisabilité matérielle et logicielle

Aucun membre de l'équipe ne dispose d'un GPU. Les premiers essais montrent toutefois que cette contrainte n'est pas bloquante : les 21 essais de modèles classiques ont demandé **22 minutes d'entraînement** au total sur un processeur ordinaire. Google Colab reste disponible en appui, avec son GPU gratuit. Le tableau suivant présente les outils retenus.

Tableau 22 *: Outils et bibliothèques retenus*

Besoin	Outil retenu	Justification
Notebooks et calcul	Jupyter, VS Code, Google Colab	Même environnement pour les trois membres, GPU gratuit si nécessaire
Partage	GitHub pour le code, Google Drive pour les données	Pas de fichiers volumineux dans Git
Données et figures	NumPy, pandas, Matplotlib, seaborn	Exploration des données et graphiques
Traitement d'images	OpenCV	Recadrage, redimensionnement et descripteurs HOG
Classifieurs	scikit-learn	SVM, Random Forest, k-NN et métriques
Extensions	XGBoost, PyTorch	Boosting et CNN entraîné de zéro, si accord
Carte	OSMnx, OpenStreetMap	Vraies rues de Casablanca
Affichage	pygame, Panda3D	Vues 2D et 3D, sans GPU dédié
Tests	pytest	Vérification du code à chaque modification

6.4- Identification des risques

Plusieurs risques ont été identifiés, et une parade a été prévue pour chacun d'eux.

Tableau 23 *: Risques identifiés et parades prévues*

Risque	Niveau	Parade
Dataset refusé par l'encadrante	Moyen	CIFAR en solution de secours, la chaîne de traitement restant identique
Scores trop faibles sur certaines classes	Élevé	Analyse des confusions et extension CNN, qui exploite la couleur
Fuite de données entre les jeux	Moyen	Découpage par photo, contrôle automatique et test ouvert une seule fois
Prétraitement différent entre notebook et simulateur	Moyen	Paramètres lus dans les métadonnées et test de concordance
Écart entre les images d'entraînement et celles du simulateur	Moyen	Caméra virtuelle fournissant de vraies photos et non des rendus 3D (section 4.4)
Intégration qui prend le pas sur le Machine Learning	Moyen	Priorité donnée au modèle, le simulateur restant dans sa version actuelle si nécessaire
Manque de temps avant la soutenance	Moyen	Modèle figé le 9 décembre, une semaine avant la soutenance

6.5- Preuves de faisabilité disponibles

Les éléments suivants démontrent que le projet est réalisable, sans pour autant signifier que l'ensemble du travail est terminé. Les scores présentés sont mesurés sur le **jeu de validation**, le jeu de test restant réservé à l'évaluation finale.

Tableau 24 *: Preuves de faisabilité disponibles au 6 novembre 2026*

Preuve	Ce qu'elle démontre	État
Étude du dataset	Les données sont disponibles et adaptées	COCO étudié et licences vérifiées (chapitre 3)
Notebook d'exploration	Les classes et leur distribution sont connues	Effectifs, tailles et exemples mesurés
Notebook de prétraitement	Les images peuvent être préparées	14 990 recadrages, découpage sans photo partagée
Premier classifieur	Le principe de classification fonctionne	SVM à noyau RBF : F1 macro de 0,591 en validation, contre 0,171 pour le hasard
Comparaison préliminaire	Les modèles peuvent être évalués	SVM, Random Forest, k-NN et XGBoost comparés (figure 16)
Architecture globale	Les modules et leurs échanges sont définis	Chapitres 2 et 4
Prototype de planification	Dijkstra trouve un chemin et en recherche un autre	Implémenté sur la carte de Casablanca et couvert par des tests automatiques
Scénario de simulation	L'intégration est techniquement plausible	Le simulateur charge un modèle et déroule les quatre scénarios (figure 15)

6.5.1- Déroulement des comparaisons préliminaires

Les comparaisons préliminaires constituent la preuve la plus directe de la faisabilité du projet. Il est donc important de préciser avec quoi, comment et pourquoi elles ont été réalisées.

Avec quoi ? Les comparaisons s'appuient sur les données préparées au chapitre 3 : **10 495 images d'entraînement** et **2 246 images de validation**, réparties de manière équilibrée entre les six classes. Le jeu de test n'a pas été utilisé. Chaque image est décrite par son descripteur **HOG** de 1 764 valeurs. Les modèles proviennent des bibliothèques scikit-learn et XGBoost, et tous les calculs ont été réalisés sur un processeur ordinaire de 12 cœurs logiques, sans GPU.

Comment ? Pour chaque famille de modèles, plusieurs réglages ont été testés. Chaque réglage a été entraîné sur le jeu d'entraînement, puis évalué sur le jeu de validation, avec les mêmes images et dans le même ordre pour tous les modèles. Pour chaque essai, nous avons relevé l'accuracy, la précision, le rappel, le F1 macro et le temps d'entraînement. Le meilleur réglage de chaque famille a ensuite été retenu selon son F1 macro, et la matrice de confusion du meilleur modèle a été analysée. Deux scores planchers servent de point de comparaison : un modèle qui répond toujours la même classe obtient un F1 macro de 0,048, et un modèle qui tire au hasard selon les fréquences obtient 0,171.

Tableau 25 *: Réglages testés lors des comparaisons préliminaires (jeu de validation)*

Famille	Réglages testés	Essais	Meilleur réglage	F1 macro	Entraînement
SVM linéaire	C de 0,001 à 1, et une variante HOG à cellules de 4 px	5	C = 0,001	0,494	34 s
SVM RBF	C = 1 ou 10	2	C = 10	**0,591**	120 s
Random Forest	100 ou 300 arbres, profondeur libre ou limitée à 20	3	300 arbres, profondeur 20	0,517	21 s
k-NN	k = 5, 11 ou 21, et PCA à 50, 100 ou 200 dimensions	9	k = 11, PCA 200	0,470	1 s
XGBoost	Profondeur 4 ou 6, jusqu'à 400 arbres	2	Profondeur 4	0,571	245 s

Au total, les **21 essais** ont demandé 22 minutes d'entraînement. Il faut noter que XGBoost s'arrête lorsque son score de validation ne progresse plus, ce qui rend son résultat légèrement optimiste. De manière générale, les scores de validation servent à choisir les réglages, et seul le jeu de test, utilisé à la fin, fournira une mesure définitive.

Pourquoi ? Ces comparaisons répondent à quatre objectifs :

- **Vérifier la faisabilité**
  Elles montrent que des modèles simples, entraînés sur nos données, reconnaissent les six classes bien mieux que le hasard.
- **Identifier la meilleure famille**
  Elles désignent le SVM à noyau RBF comme référence, que toute amélioration future devra dépasser.
- **Repérer les classes difficiles**
  La matrice de confusion du SVM permet d'identifier dès maintenant les confusions entre le camion, la voiture et le bus.
- **Estimer le coût de calcul**
  Elles confirment que l'entraînement reste réalisable sans GPU, dans des délais compatibles avec le planning.

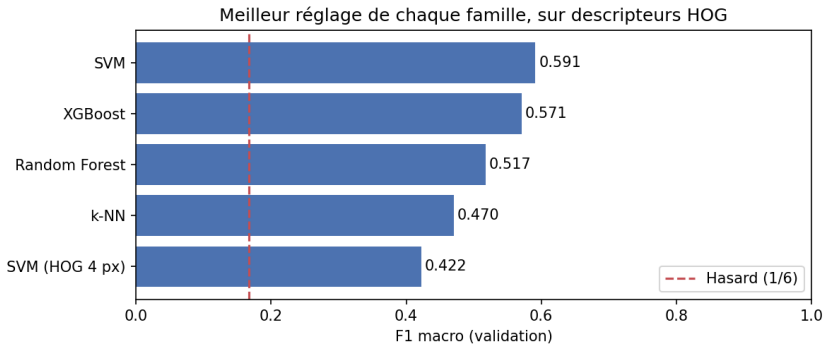


Figure 16 : *Comparaison préliminaire du meilleur réglage de chaque famille sur la validation*

Ces premiers résultats permettent de dégager plusieurs constats :

- tous les modèles dépassent nettement le hasard, représenté par la ligne rouge ;
- le **SVM à noyau RBF** arrive en tête (0,591), devant XGBoost (0,571), le Random Forest (0,517) et le k-NN (0,470) ;
- les principales confusions concernent le camion, souvent pris pour une voiture ou un bus, et le vélo, parfois pris pour une personne ou une moto ;
- un premier **CNN entraîné de zéro** atteint 0,726 en validation, cette extension restant soumise à l'accord de notre encadrante.

6.6- Planning de réalisation

Le projet s'étend sur six semaines, du 4 novembre au 16 décembre 2026. Le diagramme suivant présente la répartition chronologique des tâches. Les dates des séances et du gel du modèle sont fixes, tandis que les durées des travaux restent des estimations. Si une étape prend du retard, ce sont les étapes suivantes qui sont décalées, et jamais les séances.

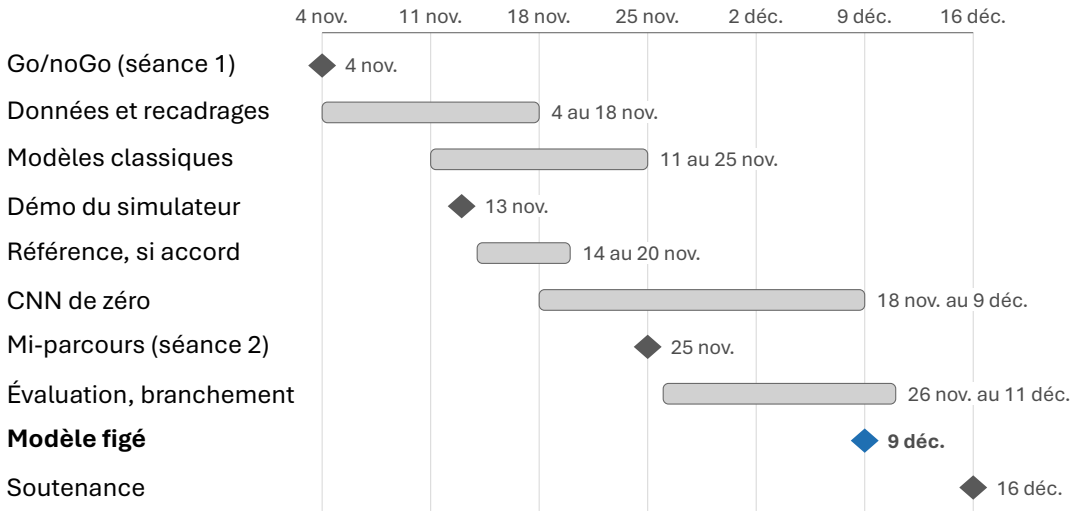


Figure 17 : *Planning prévisionnel du projet*

La figure 18 présente la répartition des efforts que nous visons. Le Machine Learning et les données en occupent la plus grande part, puisqu'ils constituent le cœur du sujet.

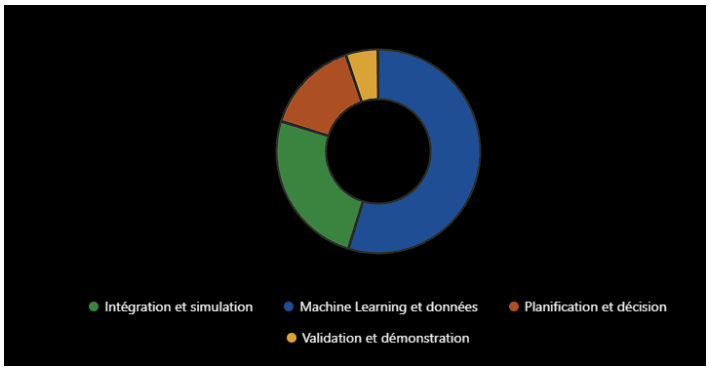


Figure 18 *: Répartition indicative des efforts (proposition et non mesure du temps déjà passé)*

6.7- Critères de décision Go/No-Go

Le livrable 1 doit surtout démontrer trois éléments : l'existence d'un dataset exploitable et d'une méthodologie réaliste, la manière dont les prédictions seront exploitées par le véhicule virtuel, et la cohérence de l'architecture qui relie la classification d'un objet à une décision de conduite. Le tableau suivant fait le point sur chacun des critères.

Tableau 26 *: Critères de décision Go/No-Go*

Critère	Constat	Statut
Un dataset exploitable existe	COCO couvre les 6 classes, et les données sont préparées	Rempli
La méthodologie est réaliste	Protocole fixé, test réservé et métriques définies	Rempli
La classification fonctionne	Tous les modèles dépassent nettement le hasard	Rempli
L'usage des prédictions est défini	Règles de décision et association prédiction-objet décrites	Rempli
L'architecture est cohérente	Modules, échanges et diagrammes définis	Rempli
Les moyens sont suffisants	Outils gratuits, entraînement possible sans GPU	Rempli
Les choix sont validés par l'encadrante	Trois points restent à confirmer	À valider

Au regard de ces éléments, nous sollicitons une **décision de Go**. Afin d'avancer dans de bonnes conditions, nous avons toutefois besoin de l'avis de notre encadrante sur trois points :

- ☐ **Jeu de données :** COCO 2017 recadré convient-il, avec CIFAR comme solution de secours ?
- ☐ **Périmètre :** XGBoost, le CNN entraîné de zéro et le simulateur peuvent-ils être acceptés comme ajouts au sujet, l'évaluation restant centrée sur SVM, Random Forest et k-NN ?
- ☐ **Modèle de référence :** pouvons-nous utiliser un ResNet-18 préentraîné uniquement comme point de comparaison, bien séparé de notre modèle ? En l'absence d'accord, il sera retiré sans que le reste du plan ne soit modifié.

6.8- Conclusion

Ce chapitre a permis d'établir la **faisabilité du projet AutoMaze AI** sur les plans technique, matériel et logiciel. Les risques identifiés disposent tous d'une parade, et les premières preuves de faisabilité confirment que les données sont exploitables et que la classification fonctionne. Le planning retenu, qui fige le modèle une semaine avant la soutenance, laisse une marge de sécurité. Ces éléments justifient notre demande de Go, sous réserve de la validation des trois points soumis à notre encadrante.

CONCLUSION GÉNÉRALE

Le projet **AutoMaze AI** s'inscrit dans une démarche de conception et de développement d'un système de **classification d'objets routiers par Machine Learning**, destiné à guider les décisions d'un véhicule autonome virtuel. À travers les différentes étapes déjà amorcées, ce travail a permis de poser des bases solides, en définissant clairement les objectifs et le périmètre du projet, en étudiant et en préparant les données, et en concevant une architecture cohérente.

Les choix effectués reposent sur le jeu de données **COCO 2017**, sur des recadrages carrés de 64 × 64 px et sur un découpage par photo qui évite toute fuite de données. Les trois classifieurs du sujet, **SVM**, **Random Forest** et **k-NN**, seront comparés sur des descripteurs HOG à l'aide du F1 macro. Les prédictions du modèle seront ensuite exploitées par un véhicule virtuel circulant dans les rues de Casablanca, la position des objets étant fournie par le simulateur et leur nature uniquement par le modèle.

Le bilan de faisabilité est encourageant. Les données sont disponibles et préparées, les premiers classifieurs dépassent nettement le hasard, et les moyens nécessaires sont entièrement gratuits. Le projet nous semble donc réalisable dans les délais, sous réserve de l'accord de notre encadrante sur les trois points présentés à la section 6.7.

Pour la suite, le travail portera sur le réglage des modèles sur la validation, puis sur l'évaluation du modèle final sur le jeu de test et sur de nouvelles images. Si le temps le permet, une attention particulière sera également accordée à l'amélioration des règles de décision, en tenant compte de la distance et de la position des objets, ainsi qu'à l'étude de l'apport d'un CNN entraîné de zéro.

En somme, ce dossier représente une **phase essentielle de préparation et de structuration** avant la réalisation complète du projet. Il constitue un socle solide pour la mise en œuvre d'un classifieur fiable et pour la démonstration de son utilité dans un contexte de conduite autonome.

Bibliographie

[1] T.-Y. Lin, M. Maire, S. Belongie et al., *Microsoft COCO : Common Objects in Context*, European Conference on Computer Vision (ECCV), 2014. https://cocodataset.org

[2] N. Dalal et B. Triggs, *Histograms of Oriented Gradients for Human Detection*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2005.

[3] C. Cortes et V. Vapnik, *Support-Vector Networks*, Machine Learning, vol. 20, 1995.

[4] L. Breiman, *Random Forests*, Machine Learning, vol. 45, 2001.

[5] T. Cover et P. Hart, *Nearest Neighbor Pattern Classification*, IEEE Transactions on Information Theory, vol. 13, 1967.

[6] T. Chen et C. Guestrin, *XGBoost : A Scalable Tree Boosting System*, ACM SIGKDD, 2016.

[7] R. Girshick, J. Donahue, T. Darrell et J. Malik, *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*, CVPR, 2014.

[8] E. W. Dijkstra, *A Note on Two Problems in Connexion with Graphs*, Numerische Mathematik, vol. 1, 1959.

[9] Contributeurs OpenStreetMap, données sous licence ODbL. https://www.openstreetmap.org

[10] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, 2011.

[11] A. Paszke et al., *PyTorch : An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.

Annexe A : Sources des illustrations

Les figures issues de nos notebooks et de notre simulateur ont été produites dans le cadre du projet. Les autres illustrations proviennent de sources externes et sont utilisées uniquement à titre d'illustration.

Tableau 27 *: Sources des illustrations*

Illustration	Source
Scénarios 1 à 4	**[Source et licence à indiquer]**
Environnements virtuels (figure 14)	**[Source et licence à indiquer]**
Détection par R-CNN (figure 1)	**[Source et licence à indiquer]**
Modèles 3D des classes	Captures de sites de modèles 3D, dont fab.com (moto) et turbosquid.com (voiture), **[auteurs à indiquer]**
Flux de développement et répartition des efforts	Schémas réalisés pour le projet
Statistiques, recadrages et comparaison des modèles	Nos notebooks, à partir de COCO 2017
Exemple d'annotation (figure 9)	Photo COCO 000000384827, publiée sur Flickr sous licence CC BY-NC 2.0, boîte tracée par nos soins
Capture du simulateur	Notre simulateur, carte © contributeurs OpenStreetMap (ODbL)

Annexe B : Usage de l'intelligence artificielle

Le simulateur et la structure des notebooks ont été préparés avec l'aide de l'outil **Claude Code**, qui a également assisté la rédaction de ce dossier. Les résultats préliminaires présentés à la section 6.5 ainsi que les chiffres du chapitre 3 proviennent de notebooks complétés et exécutés par Claude Code sur la branche `correction` du dépôt, à la demande de l'équipe. Ils sont présentés ici comme des preuves de faisabilité, et leur place dans le travail évalué reste à définir avec notre encadrante.